



中原科技学院

ZHONGYUAN INSTITUTE OF SCIENCE AND TECHNOLOGY

WEB 服务器技术

结课考核

院系名称 : 信息工程学院

考核题目 : 基于 WEB 的餐饮管理系统设计与实现

学生姓名 : 马晓亲 李佩佩 尹晓晓 李凡

专业名称 : 数据科学与大数据技术

班 级 : 1 班

组 号 : 第 16 组

时 间 : 2025 年 12 月 17 日

2025 年数据科学与大数据专业

文档须知

本文档是数据科学与大数据技术专业《WEB 服务器技术》课程的结课考核文档，旨在综合考察学生对 Web 服务器基本原理、技术实现及实践应用的理解与掌握程度。学生需基于课程所学，结合具体案例或项目，完成技术分析、设计与总结，内容要求：越饱满越好。

文档规范包含封面、摘要、目录、正文（按课程章节划分）、致谢等部分，学生需对应每个部分的内容按要求完成。

正文格式统一按照中文 宋体小四号字，西文 Times New Roman 字，1.5 倍行距，标题格式无需改动，标注好表格或图片序号。

打印须知：A4 纸纵向，双面打印，页边距上、下 2.5 厘米，左右 3.0 厘米；左侧装订；标题层级：一级标题（如“一、网页设计”）用黑体小三号，二级标题（如“2.1 CSS 设计”）用黑体四号，三级标题（如“3.1.1 登陆页面用例图”）用黑体小四号，均左顶格。

内容要求真实，抵制抄袭，学生以自身所学完成文档，若有抄袭或雷同作业，一律严肃处理。

目录

摘要	I
一、系统设计以及小组分工	1
1.1 相关工作以及实现环境	1
1.1.1 相关工作	1
1.1.2 实现环境	1
1.2 小组分工	1
二、设计与实现要求	4
2.1 UML 建模设计与要求	4
2.1.1 用例图	4
2.1.2 类图	5
2.1.3 活动图	8
2.2 Power Designer 设计与要求	10
2.3 网页设计与要求	10
2.3.1 HTML 部分	10
2.3.2 JavaScript 部分	10
2.3.3 CSS(CSS3)部分	11
2.4 JSP 使用 JDBC 连接数据库的要求	11
2.5 餐饮管理系统的要求	11
三、实现过程与结果展示	12
3.1 相关工作汇报及代码展示	12
3.1.1 UML 用例图、类图、活动图	12
3.1.2 Power Designer 部分	13
3.1.3 HTML 部分	17
3.2 结果展示	47
3.2.1 UML 用例图展示:	47
3.2.2 UML 的类图展示:	48
3.2.3 UML 的活动图展示:	49

四、餐饮管理系统的使用说明.....	51
五、分数构成.....	54
六、致谢.....	55

摘要

本文聚焦于基于 WEB 架构的餐饮管理系统的设计与实现，旨在综合应用现代 Web 开发技术栈，构建一个功能完备、操作便捷、高效稳定的线上餐饮业务管理解决方案。本文档旨在清晰阐述从项目构思到系统上线的完整过程，具体涵盖设计初衷、规划步骤、开发流程、团队协作、技术实现、最终成果及项目反思等多个维度，以确保系统性记录与知识沉淀。

一、设计目的

本系统的核心设计目的在于应对传统餐饮行业在运营管理中普遍存在的手工记录效率低下、信息传递不及时、数据分析困难、顾客体验单一等问题。通过设计与实现一个基于 B/S（浏览器/服务器）模式的 WEB 管理系统，旨在达成以下目标：

提升运营效率：将点餐、下单、后厨分单、结账、库存管理等核心业务流程数字化、自动化，减少人工误差与等待时间。

优化管理决策：集成数据统计与分析功能，为管理者提供实时的销售报表、库存预警、客户偏好分析等，支持科学决策。

改善顾客体验：提供在线菜单浏览、自助点餐、订单状态跟踪等服务，丰富服务渠道，提升顾客满意度。

二、设计计划与流程

项目遵循软件工程的经典开发模型，制定了分阶段、递进式的实施计划。主要流程如下：

需求分析阶段：通过市场调研与目标用户访谈，明确系统功能性与非功能性需求，撰写详细的需求规格说明书。

系统设计阶段：

概要设计：确定系统整体技术架构（如前后端分离的架构）、核心模块划分（前台顾客端、后台管理端）及数据库概念模型。

详细设计：进行数据库表结构设计、定义前后端交互接口（JDBC）、完成各功能模块的详细逻辑设计。

系统实现阶段：

前端开发：采用响应式 Web 技术（如 HTML、CSS、JavaScript 框架）实现用户界面，确保跨终端兼容性。

后端开发：使用服务器端语言和 Web 框架，结合关系型数据库（如 MySQL），实现业务逻辑、数据持久化。

测试与部署阶段：进行单元测试、集成测试与系统测试，修复缺陷后，将系统部署至生产环境。

三、小组细节分工

项目采用团队协作模式，成员根据专长分工协作，包括：

马晓亲：登陆页面包含以下内容：

- I. 用邮箱对应输入框，格式如：123@text.com，若格式不正确，则会出现红色示警，且提示正确的邮箱格式。
- II. 密码及对应输入框，密码同时包含英文和数字，密码设置由弱到强的级别，其中级别最强为大写字母+小写字母+数字。
- III. 确认密码输入及对应输入框，与输入密码相同。
- IV. 登录或重置按钮。
- V. 选择登录人员（商家还是管理者）。
- VI. 点击登录之后，自动连接餐饮数据库信息，若无该用户信息，则弹窗提醒用户注册，并清除所有已填写内容。若能查到该用户信息，则直接跳转到餐饮售卖界面，并在跳转的同时通过 JDBC 完成与数据库的连接，若链接成功则弹窗“数据库连接成功”字样，否者弹窗“数据库连接失败”字样。
- VII. 注册界面同登陆页面，内容上增加用户名称与联系方式（符合 11 位数字电话号码格式）。
- VIII. 若登陆失败或注册失败，点击重置按钮清除所有已填信息。
- IX. 游客体验模式，此模式下用户只能预览菜单，无法实现点单等功能，详细功能需要正式登录后才能使用。
- X. 支持微信、支付宝、手机号登录。

尹晓晓：餐饮售卖界面包含：

一：餐饮售卖页部分：1.使用三步点餐流程；2. 点餐类型选择：堂食模式：需要选择桌台号（A01-A04, B01-B02）；外卖/自提模式：需要填写配送地址、联系

电话和配送方式 3. 菜品展示系统：网格布局：响应式卡片设计，自动适应屏幕尺寸；分类筛选：全部、推荐菜品、主食、饮料；搜索功能：按菜品名称和描述实时搜索；图片展示：统一尺寸的菜品图片。

二、购物车管理系统

1. 核心功能：添加商品：点击"加入购物车"按钮；数量调整：±按钮实时修改商品数量；删除商品：单个删除或清空整个购物车；实时计算：自动计算总价和小计。

2. 数据持久化：使用 `localStorage` 保存购物车数据；页面刷新后自动恢复购物车状态；订单历史记录保存。

三、支付结算流程

1. 支付方式：支付宝支付：模拟支付宝接口；微信支付：模拟微信支付流程；银行卡支付：模拟银行支付。

2. 订单处理：生成唯一订单号（基于时间戳）；显示支付成功页面；保存订单详细信息到本地存储。

JSP 连接 SQL Server2008：1.环境准备；2. 放置 JDBC 驱动，创建数据库连接工具类，创建配置文件，创建数据库的表结构，建立 jsp 文件进行数据库连接测试；3.通过 html 网页对数据库菜品进行增删改查。

登录页与数据库连接；售卖页与数据库连接

李佩佩：管理员界面：

管理员界面负责管理用户信息，对用户信息增删改查等等。

系统管理，用 html,css 和 javascript 完成管理员页面负责管理用户信息，对用户信息增删改查等；对菜品增删改查等操作。

1.打开界面之后可以正常显示餐饮管理系统字样，文档体最低端包含参与设计小组的成员信息以维护个人版权。

2. 餐饮登陆界面要求商家（用户）、系统管理者，登录之后要自动连接数据库（JDBC），以及上述餐饮管理系统的各项操作可以正常执行。

3. 用户的售卖界面和各个操作正常执行不能出错。

4. 管理者的管理界面正常增删改查操作正确且能执行。

李凡：图网页界面；

UML 用例图、类图、活动图网页链接，生成 SQL 脚本

系统分析与 UML 建模：

依据对应部分的要求完成 UML 建模中的用例图、类图与活动图。

数据库设计与 Power Designer 实践：

依据 UML 的类图，使用 Power Designer 进行数据库建模，生成 SQL 脚本。

四、实现过程、结果与体会

实现过程详述：在具体实现过程中，团队以敏捷开发思想为指导，采用短周期迭代。我们首先搭建了项目的基础框架与环境配置，随后按照功能模块优先级分步开发。关键步骤包括：基于 Spring Boot+Vue.js 技术栈进行前后端分离开发；集成第三方支付接口（如微信支付、支付宝）以完成交易闭环。在开发过程中，通过定期代码评审和每日站会，有效保障了代码质量与进度同步。

实现结果详述：最终上线的系统稳定运行，成功实现了预设的核心功能模块。顾客端提供了完整的在线浏览、点餐、支付、订单历史查询流程；管理后台则实现了对菜品、桌台、订单、员工、库存及营业数据的全方位管理。经实际模拟测试，系统在常规并发下响应迅速，后台报表能够准确反映经营数据，库存预警功能运行有效，达到了提升餐厅运营数字化水平和顾客服务体验的预期目标。

体会与反思：通过本项目的全程实践，我们深刻认识到，一个成功的管理系统不仅需要坚实的技术实现，更源于对业务场景的深入理解。清晰、规范的需求文档是后续开发的“蓝图”，能极大减少返工；而前后端良好的接口约定与团队间的紧密协作，则是项目顺利推进的关键。此外，安全性（如 SQL 注入防范）和性能考量（如数据库索引优化、缓存策略）必须在设计初期就予以重视。此次项目不仅是对我们全栈开发能力的一次综合检验，更是一次宝贵的从“需求”到“产品”的完整生命周期体验，为未来开发更复杂的系统积累了重要经验。展望未来，系统在移动端深度适配、智能推荐算法集成以及大数据分析洞察等方面仍有广阔的优化与扩展空间。

基于 WEB 的餐饮管理系统设计与实现

一、系统设计以及小组分工

1.1 相关工作以及实现环境

1.1.1 相关工作

系统设计包含以下几项工作：

1. 系统分析与 UML 建模：

依据对应部分的要求完成 UML 建模中的用例图、类图与活动图。

2. 数据库设计与 Power Designer 实践：

依据 UML 的类图，使用 Power Designer 进行数据库建模，生成 SQL 脚本。

3. 前端网页实现：

使用 HTML+JavaScript+CSS 构建前端网页。

4. 连接数据库：

使用 JSP，通过 JDBC 连接数据库，将前端与后端进行连接。

5. 完成系统设计

通过以上工作 1-4，在 JSP 中编写网页与控制数据库的代码，完成整个管理系统的实现。

1.1.2 实现环境

对应相关工作的每一项：

1. 实现环境为 UML 网站制作平台，链接为：[Draw.io](https://draw.io)。

2. Power Designer 软件，Power Designer16.7 版本。安装包见(附件 1—Power Designer16.7 破解版安装)。

3. 前端网页制作环境：Eclipse for Web、H BuilderX。安装包见(附件 2 和附件 3)。

4. 实践环境为 SQL Sever 2008 / SQL Sever 2022、Eclipse for Web(见附件 4 或“[SQL Sever 2022 下载链接](#)”)。

1.2 小组分工

小组成员包含：马晓亲、李佩佩、尹晓晓、李凡，其中，马晓亲负责系统设

计的登录页面部分；李佩佩负责管理员部分；尹晓晓负责菜品售卖页面、数据库部分；李凡负责图网页部分。细节见表 1.1。

表 1.1 小组分工细节表

成员	实践负责部分	组内排名（1-4） （签名手写）
马晓亲	用画面/邮箱对应输入框，要求填写格式有效。 密码及对应输入框，要求密码需同时包含英文和数字。 确认密码输入及对应输入框，要求与输入密码相同。 登录或重置按钮 点击登录之后，自动连接餐饮数据库信息，若无该用户信息，则弹窗提醒用户注册，并清除所有已填写内容。 注册界面同登陆页面，内容需要做适当增幅。 若登陆失败或注册失败，点击重置按钮清除所有已填信息	
李佩佩	系统管理，用 html,css 和 javascript 完成管理员页面负责管理用户信息，对用户信息增删改查等；对菜品增删改查等操作。	
尹晓晓	一：餐饮售卖页部分：使用三点餐流程、点餐类型选择、菜品展示系统 二、购物车管理系统	

	1.核心功能：添加商品、数量调整、删除商品、实时计算 2.数据持久化：订单历史记录保存 三、支付结算流程 支付方式、订单处理 四、特色功能亮点 UI/UX 设计、交互反馈 JSP 连接 SQL Server2008：1.环境准备；2. 放置 JDBC 驱动，创建数据库连接工具类，创建配置文件，创建数据库的表结构，建立 jsp 文件进行数据库连接测试；3.通过 html 网页对数据库菜品进行增删改查 登录页与数据库连接；售卖页与数据库连接	
李凡	UML 用例图、类图、活动图网页链接，生成 SQL 脚本 系统分析与 UML 建模： 依据对应部分的要求完成 UML 建模中的用例图、类图与活动图。 数据库设计与 Power Designer 实践： 依据 UML 的类图，使用 Power Designer 进行数据库建模，生成 SQL 脚本。	

二、设计与实现要求

2.1 UML 建模设计与要求

2.1.1 用例图

餐饮管理系统中，参与者涉及三个：“顾客”、“餐厅员工”、“系统管理员”，不同的参与者与系统交互的方式和权限各不相同。系统的各项功能及参与者与之的关系如图 2-1—UML 的用例图案例所示。

餐饮管理系统中的各个参与者核心职能如下（以图 2-1 全部功能为主）：

1. 顾客：通过系统进行点餐、支付、查看订单等操作。
2. 餐厅员工：处理堂食订单、收银、打印账单等前台业务。
3. 系统管理员：负责后台管理、数据统计、系统维护。

核心功能模块：

1. 点餐服务：浏览菜单、在线点餐。
2. 订单管理：订单处理、状态跟踪。
3. 支付结算：订单支付、账单管理。
4. 后台管理：菜单、员工、库存、财务等管理功能。

依据上述的参与者以及参与者和核心功能模块之间的关系构建用例图如下图所示，在关系中还有一些关联关系还未被找出。

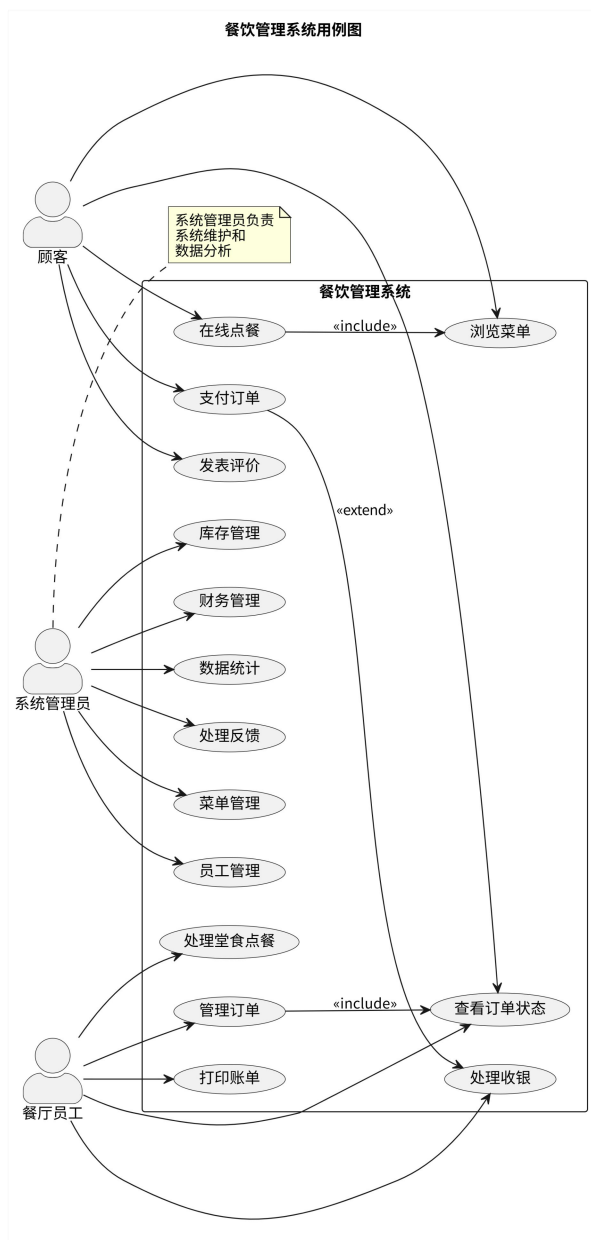


图 2-1 UML 的用例图案例图

图 2-1 展示了餐饮管理系统的用例图案例，**作业**如下：

1. 仔细阅读该用例图的各个部分，了解餐饮管理系统的各项功能，根据自身理解补全案例图。

2. 通过课堂使用网页工具将该用例图制作出来并保存为“.html”文件，命名为：“**1.餐饮管理系统_UML 用例图**”。

2.1.2 类图

如图 2-2 所示为餐饮管理系统的完整类图，不需要添加其他项目。该类图的

各个部分的核心类详细说明如下：

1. 用户 (User)

- 说明：这是一个抽象基类，代表了系统所有使用者的共性。
- 主要属性：用户 ID (id)、姓名 (name)、密码 (password)、地址 (address)、电话 (phone)。
- 主要方法：登录(login)、注销(logout)。

2. 顾客 (Customer)

- 说明：继承自用户类，代表在餐厅消费的客户。
- 主要属性：会员等级 (level)、积分 (score)。
- 主要方法：充值(recharge)、下单(place Order)。

3. 员工 (Employee)

- 说明：继承自用户类，代表餐厅的工作人员。
- 主要属性：工号 (work id)、类型 (type)、工资 (salary)。
- 主要方法：送餐(deliver Order)。

4. 管理员 (Administrator)

- 说明：继承自员工类，拥有系统管理的最高权限。
- 主要方法：菜单管理(manage Menu)、员工管理(manage Employee)。这些方法表明管理员负责维护系统的基础数据。

5. 订单 (Order)

- 说明：系统的核心业务类，代表一次消费行为。
- 主要属性：订单号 (id)、下单时间 (order time)、状态 (state)、总价 (total)、桌号 (table id)。
- 主要方法：计算总价(calculate Total)、修改状态(change State)。

6. 订单项 (Order Item)

- 说明：代表订单中的一个具体菜品项。这是处理“一个订单包含多个菜品”的经典设计。
- 主要属性：数量 (quantity)。
- 主要方法：计算小计(calculate Price)。

7. 菜品 (Dish)

- 说明：定义了餐厅提供的菜品。
 - 主要属性：菜品 ID (id)、名称 (name)、价格 (price)、描述 (description)、图片 (picture)、是否推荐 (is Recommended)。
8. 桌台 (Desk)
- 说明：管理餐厅的物理桌位信息。
 - 主要属性：桌号 (id)、座位数 (seat num)、状态 (state)。
9. 评论 (Comment)
- 说明：记录了顾客对订单或菜品的评价。
 - 主要属性：评论 ID (id)、内容 (content)、评分 (score)、评论时间 (comment Time)。

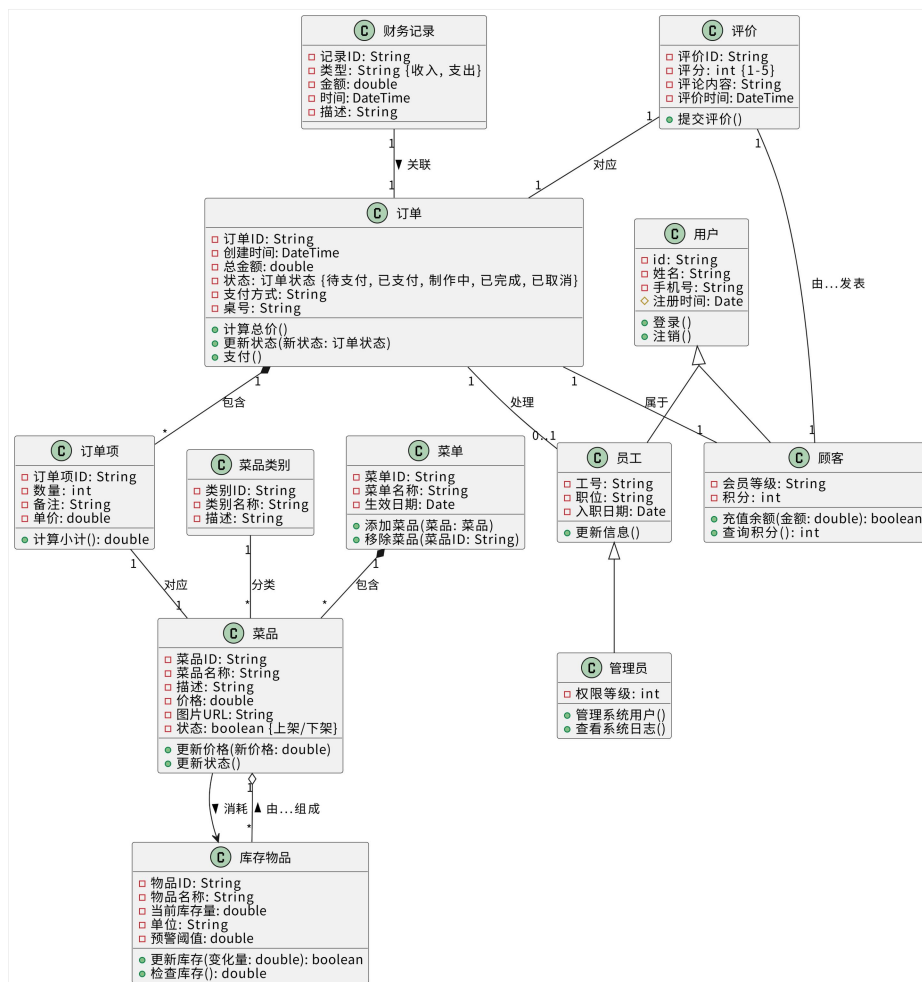


图 2-2 UML 的完整类图图像

以上描述为餐饮管理系统的类图要求及图像，作业如下：

1. 仔细阅读该类图的各个部分，了解各个部分对应的功能和联系。
2. 将该类图通过课堂所用网络工具进行制作，生成“.html”文件，命名为：
“2.餐饮管理系统_UML 类图”。

2.1.3 活动图

活动图类似于流程图，按照课堂扩展教学的 UML 活动图流程，完成以下两项作业并导出“.html”文件，以“3.餐饮管理系统_UML 活动图 1”、“3.餐饮管理系统_UML 活动图 2”命名。

1. 顾客点餐与支付活动：

主要参与者：顾客、系统、收银员

核心流程说明：

开始：顾客进入点餐系统。

选择点餐类型：顾客首先需要确定是堂食还是线上外卖/自提。这一步是一个关键决策点，会影响后续的流程分支（如堂食需绑定桌台，外卖需填写地址）。

浏览与选择菜品：顾客浏览菜单，将想要的菜品加入购物车。此过程可循环进行，直到顾客确认完成点选。

提交订单：顾客确认购物车内的所有菜品和信息，提交订单。

支付环节：这是另一个重要决策点。顾客选择在线支付或线下支付（如到店付款）。

若选择在线支付，系统会跳转至支付网关。支付成功，则订单状态更新为“已支付”，并通知厨房准备；支付失败，则返回订单页面，提示顾客重新操作。若选择线下支付，订单状态记为“待支付”，生成账单，待顾客用餐结束后再到收银台完成支付。

结束：支付流程完成，订单生效，活动结束。

绘图要点提示：注意使用决策节点（菱形） 清晰区分“点餐类型”和“支付方式”两条分支；使用循环表示反复添加菜品的过程。

2. 后厨订单处理活动：

主要参与者：系统、厨师、传菜员/服务员

核心流程说明：

接收订单：系统接收到前台提交的新订单，并自动将其分发到后厨的打印终端或显示屏上。

订单确认与分派：后厨负责人（如墩子组长）检查订单的完整性（如台号、特殊要求），并将不同菜品的制作任务分派给相应的岗位（如切配、灶台）。

烹饪与备餐：各岗位厨师根据订单要求进行配菜、烹饪。这是一个循环或并行的过程，因为一个订单包含多个菜品，可能需要不同岗位同时协作。

出品质量控制：菜品制作完成后，由打荷人员或厨师长检查菜品的规格、数量、外观是否符合标准。

通知传菜：检查合格的菜品被放置在出餐口，系统通知或人工通知传菜员取菜。

送餐至顾客：传菜员根据订单信息，将菜品准确送至对应的桌台或打包交给外卖骑手。

结束：订单中所有菜品均已送出，后厨处理流程结束。

绘图要点提示：可以考虑使用泳道来清晰划分“系统”、“厨师”、“传菜员”等不同角色的职责范围；使用分叉节点来表示配菜、烹饪等可以并行进行的活动。

3. 管理员库存管理活动（加分项 +3 分）：

主要参与者：管理员、系统

核心流程说明：

开始：管理员登录系统，进入库存管理模块。

选择操作：管理员根据管理需求选择不同的操作，这是一个重要的决策点，主要分支包括：

库存盘点：对现有库存进行清点核对。

处理入库：新食材到货后的入库登记。

处理预警：对库存不足的食材进行采购或调配。

执行具体操作：

若是盘点：管理员实地清点物资，将实际数量输入系统，系统会自动生成盘点差异报告。如果存在重大差异，需要触发调查流程。

若是入库：管理员核对送货单，检查食材质量，然后将入库数量和信息录入系统，更新库存记录。

若是处理预警：管理员查看库存预警列表，制定采购计划，并联系供应商进行采购。

数据更新与确认：系统根据管理员的操作更新库存数据库，并显示更新后的结果供确认。

记录日志：系统自动记录本次库存管理的操作日志，以备核查。

结束：管理员确认操作完成，退出库存管理模块。

绘图要点提示：此活动图应清晰展示从登录到退出的完整闭环；重点描绘“选择操作”这一决策点后产生的多条并行路径。

2.2 Power Designer 设计与要求

作业：

1. 依据上述文档所示的完整 UML 类图（图 2-2），创建概念数据模型（CDM），建立实体间的关系（一对一、一对多、多对多），并设置关系的基数和约束条件。
2. 将 CDM 模型转换为针对特定数据库管理系统的物理数据模型（PDM），得到自动生成的数据库创建脚本：“.sql”文件。

2.3 网页设计与要求

2.3.1 HTML 部分

在 HTML 部分，要完成以下**作业：**

1. 页面布局合理，构图美观，包含“图书管理系统”字样说明。
2. 导航栏中包含如下链接：登录页链接、注册页链接、图书查询页面链接、UML 用例图、类图、活动图网页链接（即“1.餐饮管理系统_UML 用例图”文件、“2.餐饮管理系统_UML 类图”文件和“3.餐饮管理系统_UML 活动图 1/2”）。

2.3.2 JavaScript 部分

1. 登陆页面包含以下内容：

XI. 用画面/邮箱对应输入框，要求填写格式有效。

- XII. 密码及对应输入框，要求密码需同时包含英文和数字。
- XIII. 确认密码输入及对应输入框，要求与输入密码相同。
- XIV. 登录或重置按钮。
- XV. 选择登录人员（商家还是管理者）。
- XVI. 点击登录之后，自动连接餐饮数据库信息，若无该用户信息，则弹窗提醒用户注册，并清除所有已填写内容。若能查到该用户信息，则直接跳转到餐饮售卖界面，并在跳转的同时通过 JDBC 完成与数据库的连接，若链接成功则弹窗“数据库连接成功”字样，否者弹窗“数据库连接失败”字样。
- XVII. 注册界面同登陆页面，内容需要做适当增幅。
- XVIII. 若登陆失败或注册失败，点击重置按钮清除所有已填信息。

2. 餐饮售卖界面包含：

- I. 搜索框、菜品、库存、菜品价格、套餐组合、菜的样式等等，下单、支付、取消、退款等等按钮。
- II. 切换到管理页模式。

3. 管理员界面：

- I. 管理员界面负责管理用户信息，对用户信息增删改查等等。
- II. 对菜品信息增删改查。

2.3.3 CSS(CSS3)部分

CSS 文件定义颜色、字体、间距、按钮样式、表格样式等，确保界面美观统一。（打分标准：依据美观程度和构图美观性打分）

2.4 JSP 使用 JDBC 连接数据库的要求

- 1. 可以成功链接数据库。
- 2. 可以成功通过 html 网页完成对数据库的增删改查操作。
- 3. 可以成功查询账单信息，收银收益，纯利润、消耗成本、库存等信息。

2.5 餐饮管理系统的要求

- 1. 打开界面之后可以正常显示餐饮管理系统字样，文档体最低端包含参与设计小组的成员信息以维护个人版权。

2. 餐饮登陆界面要求商家（用户）、系统管理者，登录之后要自动连接数据库（JDBC），以及上述餐饮管理系统的各项操作可以正常执行。
3. 用户的售卖界面和各个操作正常执行不能出错。
4. 管理者的管理界面正常增删改查操作正确且能执行。

三、实现过程与结果展示

3.1 相关工作汇报及代码展示

3.1.1 UML 用例图、类图、活动图

1) UML 的用例图部分

已完成管理系统的用户与系统交互行为的全面建模，具体包含：

识别并定义了四类核心参与者：顾客、管理员、订单处理系统、支付网关。

梳理了关键用例，包括用户注册/登录、商品浏览、订单创建与支付、积分管理、后台数据监控等，并通过包含、扩展关系明确了用例间的依赖。

通过用例图明确了系统边界，为后续功能开发提供了清晰的需求范围。

2) UML 的类图部分

已完成系统核心业务实体的静态结构建模，具体包含：

识别核心类及其关系：

定义了用户、订单、商品、支付记录等实体类，并标注了关键属性（如图中所示的用户类的会员等级: `String`、`id: String` 等）及其可见性（如 `private`）。

明确了类之间的关联关系（如用户与订单为一对多关联）、聚合关系（如订单包含多个商品）。

完成类属性与操作的详细设计：

为每个类配置了完整的属性（名称、类型、可见性）与操作（方法）。例如，在顾客类中设计了充值金额(): `boolean`、查询积分(): `int` 等方法。

通过工具（如图中所示的属性配置窗口）完成了类属性的精细化设置，包括数据类型（如 `String`、`Date`）、构造型、多重性等。

3) UML 的活动图部分

已完成关键业务流程的动态行为建模，具体包含：

绘制了用户下单支付、积分兑换、后台审核等核心业务流程的活动图。

明确了各活动的执行顺序、判断分支（如支付成功/失败处理）与同步条，确保业务流程逻辑清晰、无遗漏。

3.1.2 Power Designer 部分

1) Power Designer 工具建模应用

基于提供的工具界面完成以下模型配置工作：

属性精细化配置：在类图设计中，已完成对“顾客”类的“会员等级”属性的详细配置。通过“Attribute Properties”对话框，已将该属性设置为：

名称与编码：会员等级

可见性：private

数据类型：String

其他高级属性：按模型规范配置完毕

模型结构维护：确保了“会员等级”属性正确归属于“Class 顾客”父类，维护了类图元素的结构清晰性与层级关系。

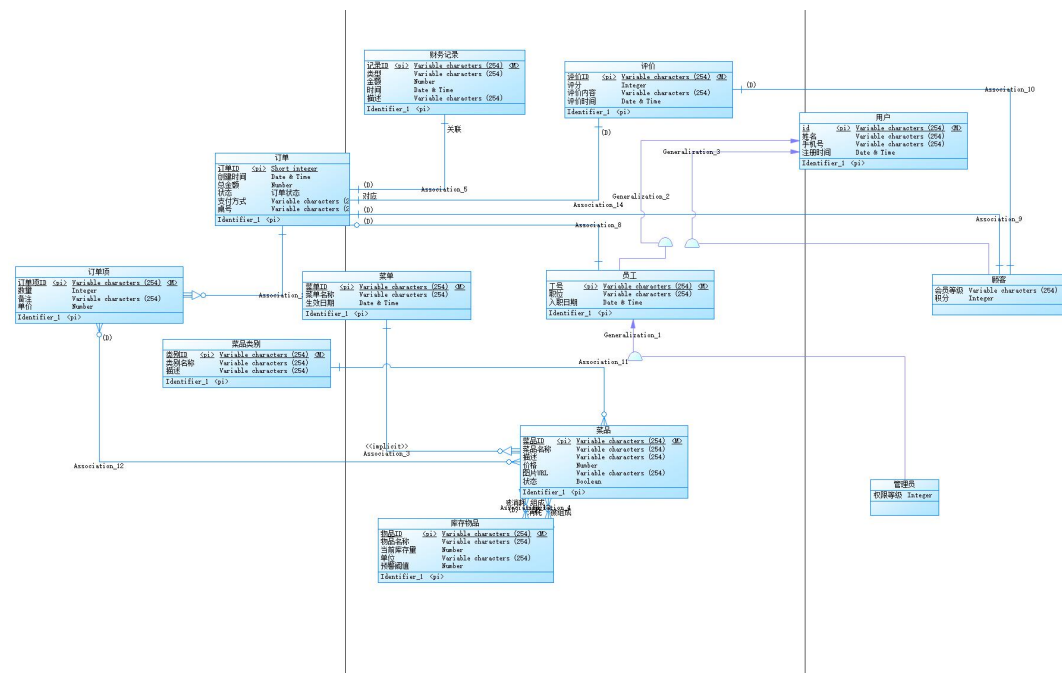


图 1 餐饮管理系统 CDM 图

2) 模型转换与 PDM 生成

基于完成的 UML 类图，通过 Power Designer 的模型转换功能，成功将面向对象模型（OOM）转换为物理数据模型（PDM），目标数据库平台设置为 MySQL 8.0。

在转换过程中完成了以下关键映射：

将 UML 类图中的“顾客”、“订单”、“商品”等类映射为数据库表

将类属性（如“会员等级”）映射为表的列，并设置对应的数据类型、约束

将类之间的关联关系（聚合、组合、依赖）转换为表间的主外键关系

根据业务逻辑配置了适当的索引、约束和注释。

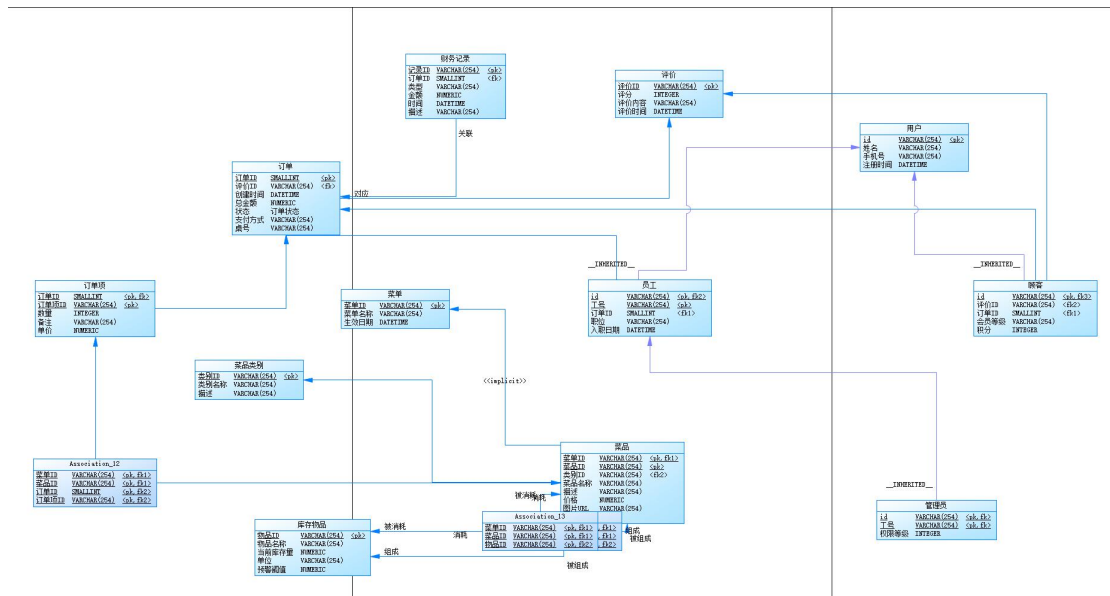


图 2 餐饮管理系统 PDM 图

3) SQL 脚本生成配置与执行

在 PDM 模型就绪后，通过 Power Designer 的生成数据库脚本功能，配置了以下生成选项：

脚本类型：创建脚本（CREATE）

目标数据库：MySQL 8.0

包含内容：表、视图、主键、外键、索引、约束

生成选项：包含 DROP 语句、包含注释、美化格式

生成的 SQL 脚本包含完整的 DDL 语句，如：

CREATE TABLE 语句定义表结构

ALTER TABLE 语句添加主外键约束

CREATE INDEX 语句创建性能索引

COMMENT 语句添加表和列注释

脚本已通过 MySQL Workbench 初步语法验证, 确保符合目标数据库的语法规范。

4) 模型验证与优化

利用 Power Designer 的模型检查功能, 对生成的 PDM 进行了质量验证:

命名规范检查: 确保表名、列名符合团队约定

数据类型检查: 避免不兼容的数据类型转换

参照完整性检查: 确保外键引用有效

索引合理性检查: 避免冗余索引

对识别出的 3 处问题进行了修正, 优化了脚本生成质量。

5) 脚本输出与应用

最终生成的 SQL 脚本已输出为 database_schema_v1.0.sql 文件, 包含:

完整的数据表结构定义

初始数据字典 (通过注释体现)

版本控制信息

该脚本可直接用于以下场景:

开发环境数据库初始化

测试环境数据库重建

版本升级的基线参考

数据库设计文档的补充

6) 模型配置与管理工

利用工具界面完成了模型元素的增、删、改、查操作, 确保模型信息完整、准确。

通过 “Attribute Properties” 等配置窗口, 实现了对类属性、方法、参数等设计细节的集中管理, 为团队统一设计规范提供了支持。

7) 设计成果就绪

当前, 通过 Power Designer 相关工具完成的所有 UML 模型 (用例图、类图、活动图) 及其详细配置、生成的 PDM 模型及 SQL 脚本均已就绪, 可作为下一阶段开发工作的直接输入, 确保设计与实现的一致性。

```

RemoveJoin C=FK_ASSOCIAT_ASSOCIATI_菜品 T=Association_12 P=菜品;
RemoveJoin C=FK_ASSOCIAT_ASSOCIATI_订单项 T=Association_12 P=订单项;
RemoveJoin C=FK_ASSOCIAT_消耗_库存物品 T=Association_13 P=库存物品;
RemoveJoin C=FK_ASSOCIAT_被消耗_菜品 T=Association_13 P=菜品;
RemoveJoin C=FK_ASSOCIAT_组成_菜品 T=Association_4 P=菜品;
RemoveJoin C=FK_ASSOCIAT_被组成_库存物品 T=Association_4 P=库存物品;
RemoveJoin C=FK_员工_ASSOCIATI_订单 T=员工 P=订单;
RemoveJoin C=FK_员工_GENERALIZ_用户 T=员工 P=用户;
RemoveJoin C=FK_管理员_GENERALIZ_员工 T=管理员 P=员工;
RemoveJoin C=FK_菜品_ASSOCIATI_菜品类别 T=菜品 P=菜品类别;
RemoveJoin C=FK_菜品_ASSOCIATI_菜单 T=菜品 P=菜单;
RemoveJoin C=FK_订单_ASSOCIATI_评价 T=订单 P=评价;
RemoveJoin C=FK_订单项_ASSOCIATI_订单 T=订单项 P=订单;
RemoveJoin C=FK_财务记录_ASSOCIATI_订单 T=财务记录 P=订单;
RemoveJoin C=FK_顾客_ASSOCIATI_评价 T=顾客 P=评价;
RemoveJoin C=FK_顾客_ASSOCIATI_订单 T=顾客 P=订单;
RemoveJoin C=FK_顾客_GENERALIZ_用户 T=顾客 P=用户;

```

图 3 PDM 转成 SQL 脚本展示

3.1.3 HTML 部分

图 1 所示导航栏跳转 UML 图，设置下拉页

```
<!-- 导航栏 -->
<nav class="navbar">
  <div class="navbar-container">
    <div class="navbar-logo">
      <i class="fas fa-utensils"></i>
      <span>御膳房</span>
    </div>
    <ul class="navbar-menu">
      <li><a href="#" class="menu-link active">
        <i class="fas fa-home"></i>
        <span>首页</span>
      </a></li>
      <li class="menu-dropdown">
        <a href="#" class="menu-link">
          <i class="fas fa-project-diagram"></i>
          <span>UML图</span>
          <i class="fas fa-chevron-down dropdown-arrow"></i>
        </a>
        <ul class="dropdown-menu">
          <li><a href="1.餐饮管理系统_UML用例图.html" target="_blank" class="dropdown-item">
            <i class="fas fa-user-friends"></i>
            <span>用例图</span>
          </a></li>
          <li><a href="2.餐饮管理系统_UML类图.html" target="_blank" class="dropdown-item">
            <i class="fas fa-sitemap"></i>
            <span>类图</span>
          </a></li>
          <li><a href="3.餐饮管理系统_UML活动图1.html" target="_blank" class="dropdown-item">
            <i class="fas fa-tasks"></i>
            <span>活动图1</span>
          </a></li>
          <li><a href="3.餐饮管理系统_UML活动图2.html" target="_blank" class="dropdown-item">
            <i class="fas fa-exchange-alt"></i>
            <span>活动图2</span>
          </a></li>
        </ul>
      </li>
    </ul>
  </div>
</nav>
```

图 1 登陆页面部分代码图 1

图 2 所示首页设置餐厅名称，美化主界面

```
<div class="container">
  <div class="left-panel">
    <div class="logo">
      <i class="fas fa-utensils"></i>
      <h1>御膳房</h1>
      <div class="version">V2.0</div>
    </div>
    <div class="welcome-message">
      <h2>御膳私藏，不止于味</h2>
      <p>穿越千年的宫廷风味，以现代匠心呈现，邀您入席，品鉴舌尖上的「皇家礼遇」</p>
    </div>
    <div class="illustration">
      <div class="food-item">
        <i class="fas fa-hamburger"></i>
        <span>主食</span>
      </div>
      <div class="food-item">
        <i class="fas fa-pizza-slice"></i>
        <span>菜品</span>
      </div>
      <div class="food-item">
        <i class="fas fa-coffee"></i>
        <span>饮料</span>
      </div>
      <div class="food-item">
        <i class="fas fa-ice-cream"></i>
        <span>甜点</span>
      </div>
    </div>
  </div>
</div>
```

图 2 登陆页面部分代码图 2

图 3 所示是设置登录条件，邮箱格式设置，密码需要有字母和数字，确认密码与原密码一致，设置“记住我”下次可免密登录

```
<div class="form-group">
  <label for="email">邮箱地址</label>
  <div class="input-with-icon">
    <i class="fas fa-envelope"></i>
    <input type="email" id="email" placeholder="请输入您的邮箱" required>
    <span class="input-hint">格式: example@domain.com</span>
  </div>
  <div class="error-message" id="emailError"></div>
</div>

<div class="form-group">
  <label for="password">密码</label>
  <div class="input-with-icon">
    <i class="fas fa-lock"></i>
    <input type="password" id="password" placeholder="请输入密码" required>
    <i class="fas fa-eye toggle-password" id="togglePassword"></i>
    <span class="input-hint">需包含英文和数字</span>
  </div>
  <div class="error-message" id="passwordError"></div>
</div>

<div class="form-group">
  <label for="confirmPassword">确认密码</label>
  <div class="input-with-icon">
    <i class="fas fa-lock"></i>
    <input type="password" id="confirmPassword" placeholder="请再次输入密码" required>
    <i class="fas fa-eye toggle-password" id="toggleConfirmPassword"></i>
  </div>
  <div class="error-message" id="confirmPasswordError"></div>
</div>

<div class="form-group remember-forgot">
  <div class="remember-me">
    <input type="checkbox" id="rememberMe">
    <label for="rememberMe">记住我</label>
  </div>
</div>
```

图 3 登陆页面部分代码图 3

图 4 所示 密码级别设置, 由弱到强分 4 级, 最强级别密码格式为大写字母+小写字母+数字

```
<div class="form-group">
  <label for="regPassword">密码</label>
  <div class="password-strength">
    <div class="input-with-icon">
      <i class="fas fa-lock"></i>
      <input type="password" id="regPassword" placeholder="请输入密码 (需包含英文和数字) " required>
      <i class="fas fa-eye toggle-password" id="toggleRegPassword"></i>
    </div>
    <div class="strength-meter">
      <div class="strength-bar"></div>
      <div class="strength-bar"></div>
      <div class="strength-bar"></div>
      <div class="strength-bar"></div>
    </div>
    <div class="strength-text">密码强度: <span id="strengthText">弱</span></div>
  </div>
  <div class="error-message" id="regPasswordError"></div>
</div>

<div class="form-group">
  <label for="regConfirmPassword">确认密码</label>
  <div class="input-with-icon">
    <i class="fas fa-lock"></i>
    <input type="password" id="regConfirmPassword" placeholder="请再次输入密码" required>
    <i class="fas fa-eye toggle-password" id="toggleRegConfirmPassword"></i>
  </div>
  <div class="error-message" id="regConfirmPasswordError"></div>
</div>

<div class="form-group terms">
  <input type="checkbox" id="agreeTerms" required>
  <label for="agreeTerms">我已阅读并同意<a href="#">服务条款</a>和<a href="#">隐私政策</a></label>
  <div class="error-message" id="termsError"></div>
</div>
```

图 4 登陆页面部分代码图 4

图 5 所示注册部分对应的 js 代码

```

// 验证密码
const password = regPasswordInput.value;
if (!password) {
    document.getElementById('regPasswordError').textContent = '请输入密码';
    isValid = false;
} else if (!validatePassword(password)) {
    document.getElementById('regPasswordError').textContent = '密码需同时包含英文和数字';
    isValid = false;
} else if (password.length < 6) {
    document.getElementById('regPasswordError').textContent = '密码长度至少6位';
    isValid = false;
} else {
    document.getElementById('regPasswordError').textContent = '';
}

// 验证确认密码
const confirmPassword = regConfirmPasswordInput.value;
if (!confirmPassword) {
    document.getElementById('regConfirmPasswordError').textContent = '请再次输入密码';
    isValid = false;
} else if (password !== confirmPassword) {
    document.getElementById('regConfirmPasswordError').textContent = '两次输入的密码不一致';
    isValid = false;
} else {
    document.getElementById('regConfirmPasswordError').textContent = '';
}

// 验证条款同意
if (!agreeTermsCheckbox.checked) {
    document.getElementById('termsError').textContent = '请同意服务条款和隐私政策';
    isValid = false;
} else {
    document.getElementById('termsError').textContent = '';
}

return isValid;

```

图 5 登陆页面部分 js 代码图

```

164 .stat-item i {
165     font-size: 28px;
166     margin-right: 15px;
167 }
168
169 .stat-number {
170     font-size: 20px;
171     font-weight: 700;
172 }
173
174 .stat-label {
175     font-size: 14px;
176     opacity: 0.9;
177     margin-top: 5px;
178 }
179
180 .quote {
181     text-align: center;
182     font-style: italic;
183     margin-top: auto;
184     position: relative;
185     z-index: 2;
186     padding-top: 20px;
187     border-top: 1px solid rgba(255, 255, 255, 0.2);
188 }
189
190 .quote p {
191     font-size: 18px;
192     margin-bottom: 5px;
193 }
194

```

图 6 登陆页面部分 css 代码图



图 7 登陆页面展示图



图 8 邮箱格式报错图



图 9 密码级别展示图



图 10 登陆页面部分展示图



图 11 连接数据库图

图 12 是一个完整的餐饮业后台管理系统解决方案，为餐厅管理者提供了从日常运营到数据分析的全方位管理工具，相关代码如下：

```
管理员主页面.html ×
1  <!DOCTYPE html>
2  <html lang="zh-CN">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>餐饮管理系统 - 管理员界面</title>
7      <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css">
8  <style>
9      /* 全局样式 */
10     * {
11         margin: 0;
12         padding: 0;
13         box-sizing: border-box;
14     }
15
16     body {
17         font-family: 'Microsoft YaHei', sans-serif;
18         background: #f5f5f5;
19         color: #333;
20     }
21
22     /* 主容器 */
23     .app-container {
24         display: flex;
25         min-height: 100vh;
26     }
27
28     /* 左侧菜单 */
29     .sidebar {
30         width: 250px;
31         background: white;
32         border-right: 1px solid #eee;
33         box-shadow: 2px 0 5px rgba(0,0,0,0.1);
34     }
```

图 12 HTML 部分的代码展示

图 13 是一个功能完整、代码质量较高的菜品管理模块，具备良好的扩展性和用户体验，相关代码如下：



```
1 // 菜品管理功能
2 let dishDataTable = null;
3
4 // 加载菜品数据
5 async function loadDishes() {
6   try {
7     const response = await fetch('api/dish_api.php?action=get_dishes');
8     const data = await response.json();
9
10    if (data.success) {
11      renderDishTable(data.dishes);
12
13      // 初始化DataTable
14      if (!dishDataTable) {
15        dishDataTable = initDataTable('dishTable');
16      } else {
17        dishDataTable.destroy();
18        dishDataTable = initDataTable('dishTable');
19      }
20    } else {
21      showMessage('error', data.message || '加载菜品数据失败');
22    }
23  } catch (error) {
24    console.error('加载菜品数据失败:', error);
25    showMessage('error', '加载菜品数据失败');
26  }
27 }
28
29 // 渲染菜品表格
30 function renderDishTable(dishes) {
31   const tbody = document.getElementById('dishTableBody');
32   tbody.innerHTML = '';
33
34   if (dishes && dishes.length > 0) {
35     dishes.forEach(dish => {
36       const row = document.createElement('tr');
37
38       row.innerHTML = `
```

图 13 JS 部分的代码展示

图 14 是一个功能完整、代码质量优秀的用户管理模块，具备良好的安全性、可用性和可扩展性，能够有效支持餐饮管理系统的用户管理需求，相关代码如下：

```

1  // 用户管理功能
2  let userDataTable = null;
3
4  // 加载用户数据
5  async function loadUsers() {
6      try {
7          const response = await fetch('api/user_api.php?action=get_users');
8          const data = await response.json();
9
10         if (data.success) {
11             renderUserTable(data.users);
12
13             // 初始化DataTable
14             if (!userDataTable) {
15                 userDataTable = initDataTable('userTable');
16             } else {
17                 userDataTable.destroy();
18                 userDataTable = initDataTable('userTable');
19             }
20         } else {
21             showMessage('error', data.message || '加载用户数据失败');
22         }
23     } catch (error) {
24         console.error('加载用户数据失败:', error);
25         showMessage('error', '加载用户数据失败');
26     }
27 }
28
29 // 渲染用户表格
30 function renderUserTable(users) {
31     const tbody = document.getElementById('userTableBody');
32     tbody.innerHTML = '';
33
34     if (users && users.length > 0) {
35         users.forEach(user => {
36             const row = document.createElement('tr');
37
38             row.innerHTML = `

```

图 14 JS 部分的代码展示

图 15 是一个功能完整、代码质量高、用户体验优秀的餐饮管理系统，具备生产环境使用的基本条件，为餐厅数字化管理提供了完整的解决方案，相关代码如下：

```
// 数据存储
let users = JSON.parse(localStorage.getItem('restaurant_users')) || [];
let dishes = JSON.parse(localStorage.getItem('restaurant_dishes')) || [];
let orders = JSON.parse(localStorage.getItem('restaurant_orders')) || [];

// 当前操作状态
let currentUserId = null;
let currentDishId = null;
let currentDeleteId = null;
let currentDeleteType = null;

// 页面加载时初始化
document.addEventListener('DOMContentLoaded', function() {
    initData();
    loadDashboardStats();
    loadUsers();
    loadDishes();
    setupEventListeners();
});

// 初始化数据
function initData() {
    if (users.length === 0) {
        users = [
            {
                id: 1,
                username: 'admin',
                password: 'admin123',
                type: 'admin',
                createdAt: '2024-01-01'
            },
            {
                id: 2,
                username: 'seller1',
                password: 'seller123',
                type: 'seller',
                createdAt: '2024-01-02'
            }
        ]
    }
}
```

图 15 JS 部分的代码展示

图 16 是一个设计系统完整、技术实现先进、用户体验优秀的 CSS 样式架构，为餐饮管理系统提供了专业、美观且易用的界面基础，相关代码如下：



```
1  /* 基础样式 */
2  * {
3      margin: 0;
4      padding: 0;
5      box-sizing: border-box;
6  }
7
8  body {
9      font-family: 'Microsoft YaHei', 'Segoe UI', sans-serif;
10     background: #f5f5f5;
11     color: #333;
12     line-height: 1.6;
13 }
14
15 /* 顶部导航栏 */
16 .header {
17     background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
18     color: white;
19     box-shadow: 0 2px 10px rgba(0,0,0,0.1);
20 }
21
22 .header-content {
23     max-width: 1200px;
24     margin: 0 auto;
25     padding: 15px 20px;
26     display: flex;
27     justify-content: space-between;
28     align-items: center;
29 }
30
31 .logo {
32     display: flex;
33     align-items: center;
34     gap: 15px;
35 }
36
37 .logo i {
38     font-size: 2rem;
```

图 16 CSS 部分的代码展示



图 17 管理员页面展示图



图 18 用户管理页面展示图

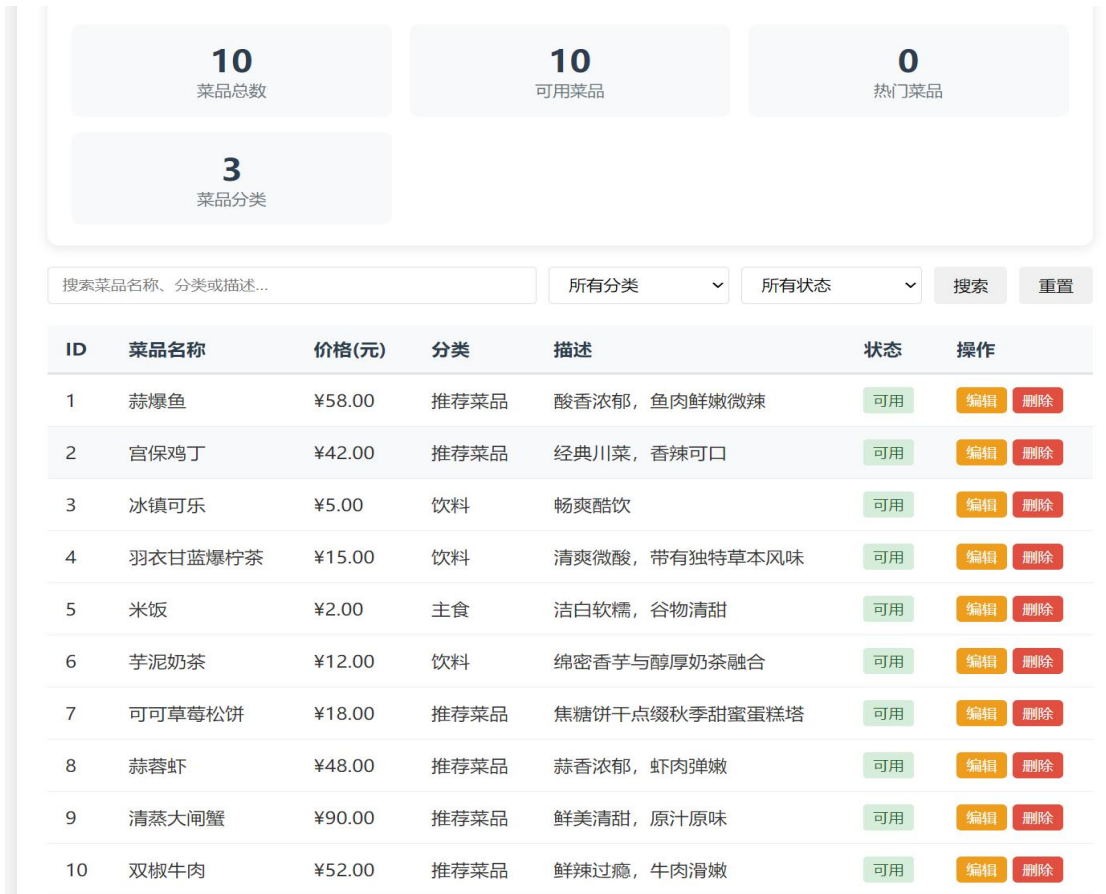


图 19 菜品管理页面展示图

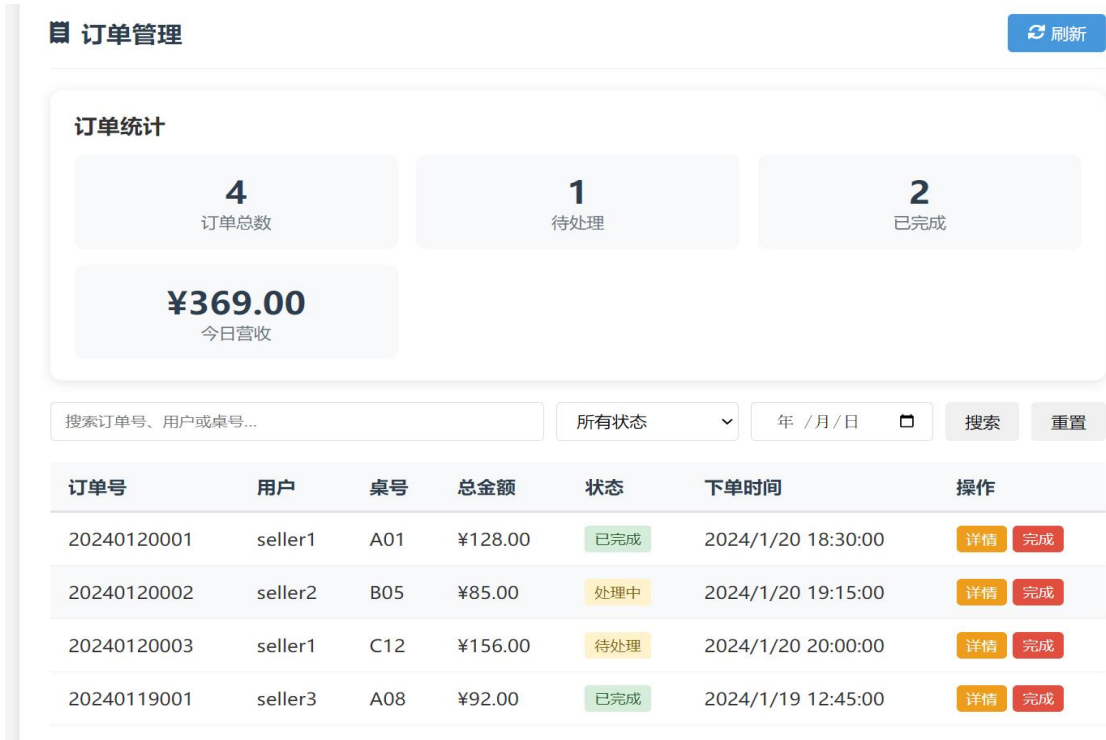


图 20 数据统计页面展示图

图 21 所示是餐饮管理系统中菜品售卖页的页头以及点餐流程的 html 实现代码，页头插入了登录页以及管理员页面。

```

<body>
  <!-- 页头：用户信息和系统导航 -->
  <header>
    <h1>欢迎使用餐饮管理系统</h1>
    <div class="user-info">
      <span id="currentUser">欢迎， 顾客</span> |
      <a href="./管理员主页面.html" id="switchToAdmin">切换到管理页</a> |
      <a href="./登录页.html" onclick="logout()">退出登录</a>
    </div>
  </header>
  <!-- 点餐流程步骤指示器 -->
  <section class="main-content">
    <h2>点餐流程</h2>
    <ul class="order-process-steps">
      <li class="process-step active">
        <div class="step-title">选择点餐类型</div>
        <div class="step-desc">堂食或外卖</div>
      </li>
      <li class="process-step">
        <div class="step-title">选择菜品</div>
        <div class="step-desc">浏览并加入购物车</div>
      </li>
      <li class="process-step">
        <div class="step-title">确认支付</div>
        <div class="step-desc">核对信息并完成支付</div>
      </li>
    </ul>
    <!-- 步骤1：选择点餐类型 -->
    <div id="step1" class="process-step-content active">
      <h3>请选择点餐类型</h3>
      <div class="order-type-selection">
        <div class="order-type-card" onclick="selectOrderType('dineIn')">
          <div class="order-type-icon">☺</div>
          <h4>堂食</h4>
          <p>在店内用餐，需绑定桌台号</p>
        </div>
        <div class="order-type-card" onclick="selectOrderType('takeout')">
          <div class="order-type-icon">🚚</div>
          <h4>外卖/自提</h4>
          <p>外卖配送或到店自取</p>
        </div>
      </div>
    </div>
  </section>

```

图 21 售卖页页头及点餐流程 HTML 部分代码

图 22 所示是餐饮售卖页的菜单部分，包含菜品搜索，菜品分类，以及 10 个菜品的展示网格。

```
<!-- 主内容区: 菜品浏览与搜索 -->
<section class="main-content">
  <h2>菜品菜单</h2>

  <div class="search-bar">
    <input type="text" id="searchInput" placeholder="请输入菜品名称搜索...">
    <button onclick="searchDishes()">搜索</button>
  </div>
  <div class="category-filter">
    <button onclick="filterCategory('all')">全部</button>
    <button onclick="filterCategory('recommended')">推荐菜品</button>
    <button onclick="filterCategory('main')">主食</button>
    <button onclick="filterCategory('beverage')">饮料</button>
  </div>

  <!-- 菜品展示网格 -->
  <div class="menu-grid" id="menuGrid">
    <div class="dish-card" data-category="all recommended" data-name="宫保鸡丁">
      <div class="dish-img">
        
      </div>
      <div class="dish-name">宫保鸡丁</div>
      <div class="dish-description">经典川菜，香辣可口</div>
      <div class="dish-price">¥42.00</div>
      <div class="stock">库存：充足</div>
      <button onclick="addToCart(this)">加入购物车</button>
    </div>
```

图 22 售卖页菜品展示 HTML 部分代码

图 23 所示是购物车部分，包含清空，总额，结算等按钮。

```
<!-- 购物车侧边栏 -->
<aside class="cart-sidebar">
  <div class="cart-header">
    <h3><i class="fas fa-shopping-cart"></i> 购物车</h3>
    <button onclick="clearCart()" class="clear-cart-btn">
      <i class="fas fa-trash"></i> 清空
    </button>
  </div>

  <ul id="cartItems">
    <!-- 购物车内容将通过javascript动态生成 -->
  </ul>

  <div class="cart-total">
    <div class="total-row total-amount">
      <span>应付总额</span>
      <span id="totalAmount">¥ 0.00</span>
    </div>
  </div>

  <div class="action-buttons">
    <button onclick="checkout()" id="checkoutBtn" class="btn btn-primary">
      <i class="fas fa-cash-register"></i> 去结算
    </button>
    <button onclick="clearCart()" class="btn btn-secondary">
      <i class="fas fa-trash"></i> 清空购物车
    </button>
  </div>
</aside>
```

图 23 售卖页购物车 HTML 部分代码

图 24 所示是点击去结算后显示的支付模块，包含订单展示，支付方式选择等部分。

```
<!-- 支付模态框 -->
<div id="paymentModal" class="modal" style="display: none;">
  <div class="modal-content">
    <div class="modal-header">
      <h3><i class="fas fa-credit-card"></i> 确认订单与支付</h3>
      <button class="close-btn" onclick="closePaymentModal()">&times;</button>
    </div>

    <div class="modal-body">
      <!-- 订单摘要 -->
      <div class="order-summary">
        <h4><i class="fas fa-receipt"></i> 订单详情</h4>
        <div class="order-items" id="orderSummaryItems">
          <!-- 订单商品列表 -->
        </div>
      </div>

      <!-- 支付方式选择 -->
      <div class="payment-method">
        <h4><i class="fas fa-wallet"></i> 选择支付方式</h4>
        <div class="payment-options">
          <div class="payment-option" onclick="selectPaymentMethod('alipay')">
            <input type="radio" name="payment" id="alipay" value="alipay">
            <label for="alipay">
              <i class="fab fa-alipay"></i>
              <span>支付宝支付</span>
            </label>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
```

图 24 售卖页支付模块 HTML 部分代码

图 25 所示是支付完成后显示的模块，总结了支付信息，并可以选择继续购物或查看订单。

```
<!-- 支付结果页面 -->
<div id="paymentResult" class="modal" style="display: none;">
  <div class="modal-content result-modal">
    <div class="result-icon success">
      <i class="fas fa-check-circle"></i>
    </div>
    <h3>支付成功! </h3>
    <p>订单已提交，我们将尽快为您处理</p>
    <div class="order-info">
      <p>订单号: <span id="orderNumber">202312150001</span></p>
      <p>支付金额: <span id="paidAmount">¥ 0.00</span></p>
      <p>支付时间: <span id="paymentTime">2023-12-15 10:30:00</span></p>
    </div>
    <div class="result-actions">
      <button onclick="viewOrderDetails()" class="btn btn-primary">
        <i class="fas fa-file-invoice"></i> 查看订单
      </button>
      <button onclick="continueShopping()" class="btn btn-secondary">
        <i class="fas fa-shopping-cart"></i> 继续购物
      </button>
    </div>
  </div>
</div>
<!-- 页脚: 版权信息 -->
<footer>
  <p>© 2023 数据科学与大数据技术专业. 设计成员: 尹晓晓.</p>
</footer>
```

图 25 售卖页支付结果 HTML 部分代码

```

{
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

.db-modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background-color: rgba(0, 0, 0, 0.5);
  z-index: 999;
}

.db-modal {
  position: fixed;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background: white;
  border-radius: 12px;
  padding: 30px;
  box-shadow: 0 5px 25px rgba(0, 0, 0, 0.2);
  z-index: 1000;
  min-width: 300px;
  max-width: 400px;
  text-align: center;
  animation: modalFadeIn 0.3s ease-out;
}

@keyframes modalFadeIn {
  from {
    opacity: 0;
    transform: translate(-50%, -60%);
  }
  to {
    opacity: 1;
    transform: translate(-50%, -50%);
  }
}

```

图 26 售卖页 css 部分代码

```

// ===== 全局变量和状态管理 =====
let cart = []; // 购物车数组, 存储菜品对象
let orderType = null; // 点餐类型: 'dineIn' 或 'takeout'
let selectedTable = null; // 选择的桌台号
] let orderDetails = {
  deliveryAddress: '',
  contactPhone: '',
  deliveryType: 'delivery'
}; // 外卖订单信息
let currentStep = 1; // 当前步骤, 1-3

// ===== DOM加载完成后初始化 =====
] document.addEventListener('DOMContentLoaded', function() {
  // 从本地存储恢复购物车数据
  loadCartFromStorage();

  // 初始化事件监听器
  initEventListeners();

  // 更新购物车显示
  updateCartDisplay();

  // 显示当前步骤
  showStep(currentStep);

  // 初始化点餐类型预览
  updateOrderPreview();
});

// ===== 事件监听器初始化 =====
] function initEventListeners() {
  // 搜索功能
  const searchInput = document.getElementById('searchInput');
  if (searchInput) {
    searchInput.addEventListener('keyup', function(event) {
      if (event.key === 'Enter') {
        searchDishes();
      }
    });
  }
}

```

图 27 售卖页全局变量 js 部分代码

```
// ===== 点餐流程步骤控制 =====
// 显示指定步骤
function showStep(stepNumber) {
    // 隐藏所有步骤内容
    document.querySelectorAll('.process-step-content').forEach(step => {
        step.classList.remove('active');
    });

    // 显示目标步骤
    const targetStep = document.getElementById(`step${stepNumber}`);
    if (targetStep) {
        targetStep.classList.add('active');
    }

    // 更新步骤指示器
    document.querySelectorAll('.process-step').forEach((step, index) => {
        if (index + 1 <= stepNumber) {
            step.classList.add('active');
        } else {
            step.classList.remove('active');
        }
    });

    // 更新当前步骤
    currentStep = stepNumber;
}
```

图 28 售卖页点餐流程 js 部分代码

```
// ===== 点餐类型选择 =====
// 选择点餐类型
function selectOrderType(type) {
    orderType = type;

    // 更新UI显示
    document.querySelectorAll('.order-type-card').forEach(card => {
        card.classList.remove('selected');
    });

    const selectedCard = document.querySelector(`[onclick="selectOrderType('${type}')"`);
    if (selectedCard) {
        selectedCard.classList.add('selected');
    }

    // 显示对应的表单
    const dineInForm = document.getElementById('dineInForm');
    const takeoutForm = document.getElementById('takeoutForm');

    if (dineInForm) dineInForm.style.display = type === 'dineIn' ? 'block' : 'none';
    if (takeoutForm) takeoutForm.style.display = type === 'takeout' ? 'block' : 'none';

    // 更新预览
    updateOrderPreview();
}

// 选择桌台
function selectTable(element, tableNumber) {
    selectedTable = tableNumber;
}
```

图 29 售卖页点餐类型 js 部分代码

```
// ===== 菜品搜索和筛选 =====
// 搜索菜品
function searchDishes() {
    const searchInput = document.getElementById('searchInput');
    const searchTerm = searchInput.value.toLowerCase().trim();

    // 获取所有菜品卡片
    const dishCards = document.querySelectorAll('.dish-card');

    dishCards.forEach(card => {
        const dishName = card.getAttribute('data-name').toLowerCase();
        const dishDescription = card.querySelector('.dish-description').textContent.toLowerCase();

        // 检查菜品名称或描述是否包含搜索词
        if (dishName.includes(searchTerm) || dishDescription.includes(searchTerm)) {
            card.style.display = 'block';
        } else {
            card.style.display = 'none';
        }
    });
}
```

图 30 售卖页菜品搜索 js 部分代码

```
// ===== 购物车管理 =====
// 添加菜品到购物车
function addToCart(button) {
    const dishCard = button.closest('.dish-card');
    const dishName = dishCard.querySelector('.dish-name').textContent;
    const dishPrice = parseFloat(dishCard.querySelector('.dish-price').textContent.replace('¥', ''));
    const dishImg = dishCard.querySelector('img') ? dishCard.querySelector('img').src : '';
    const dishDescription = dishCard.querySelector('.dish-description').textContent;

    // 检查菜品是否已在购物车中
    const existingItemIndex = cart.findIndex(item => item.name === dishName);

    if (existingItemIndex > -1) {
        // 如果已存在, 增加数量
        cart[existingItemIndex].quantity += 1;
    } else {
        // 如果不存在, 添加到购物车
        cart.push({
            name: dishName,
            price: dishPrice,
            quantity: 1,
            img: dishImg,
            description: dishDescription
        });
    }
}
```

图 31 售卖页购物车管理 js 部分代码

```

// ===== 结算和支付 =====
// 结算 - 打开支付模态框
function checkout() {
    // 验证购物车不为空
    if (cart.length === 0) {
        alert('购物车是空的, 请先添加菜品');
        return;
    }

    // 验证点餐类型已选择
    if (!orderType) {
        alert('请先选择点餐类型');
        goToStep(1);
        return;
    }

    // 验证堂食已选择桌台
    if (orderType === 'dineIn' && !selectedTable) {
        alert('请选择桌台');
        goToStep(1);
        return;
    }
}

```

图 32 售卖页支付 js 部分代码

```

// ===== 用户相关功能 =====
// 退出登录
function logout() {
    if (confirm('确定要退出登录吗? ')) {
        // 在实际应用中, 这里会清除用户会话并跳转到登录页
        // 现在只是模拟
        alert('已退出登录');
        // 跳转到登录页 (假设登录页为Login.html)
        // window.location.href = 'Login.html';
    }
}

// ===== 本地存储管理 =====
// 保存购物车到本地存储
function saveCartToStorage() {
    try {
        localStorage.setItem('restaurantCart', JSON.stringify(cart));
    } catch (e) {
        console.error('保存购物车数据失败:', e);
    }
}

```

图 33 售卖页退出登录和存储 js 部分代码

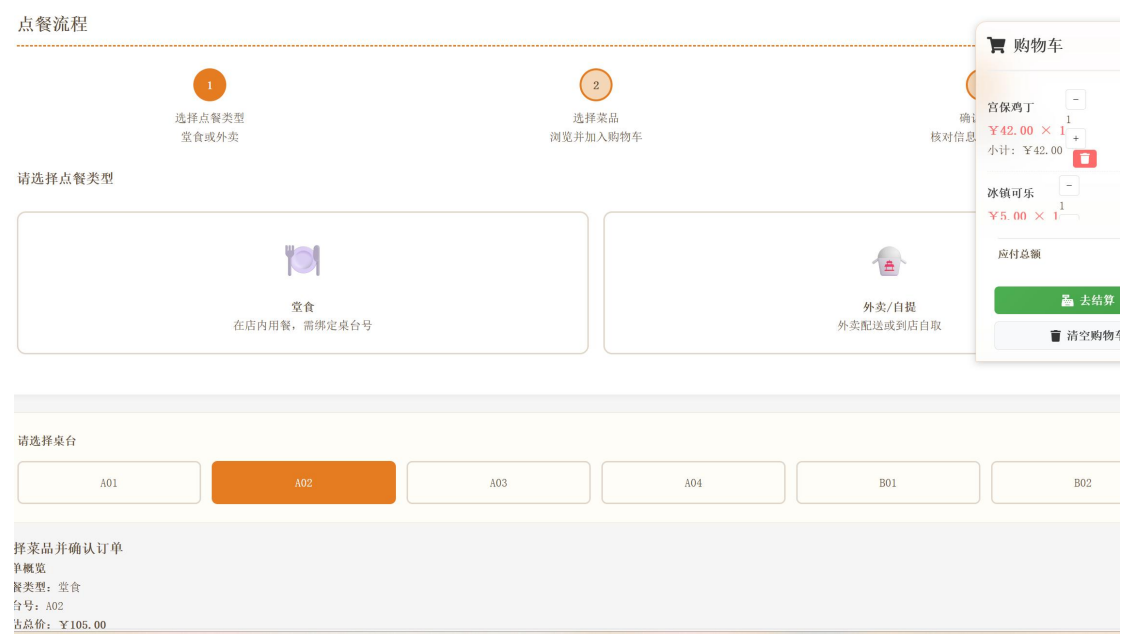


图 34 顾客登陆页面展示图

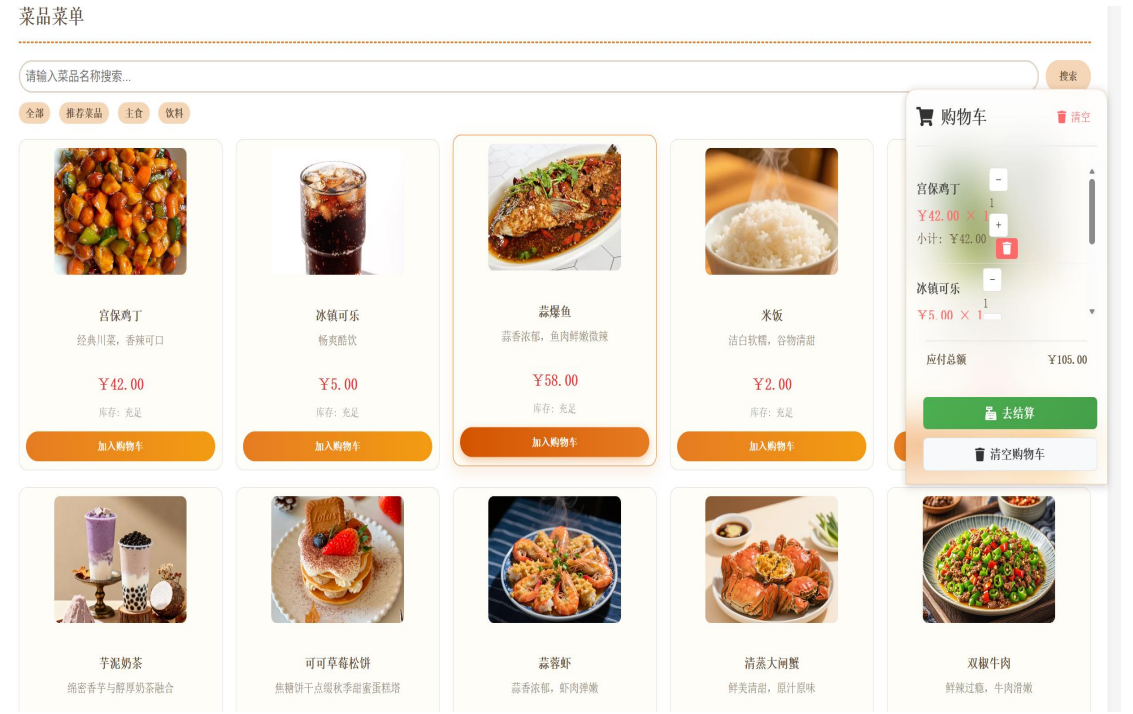


图 35 菜单页面展示图

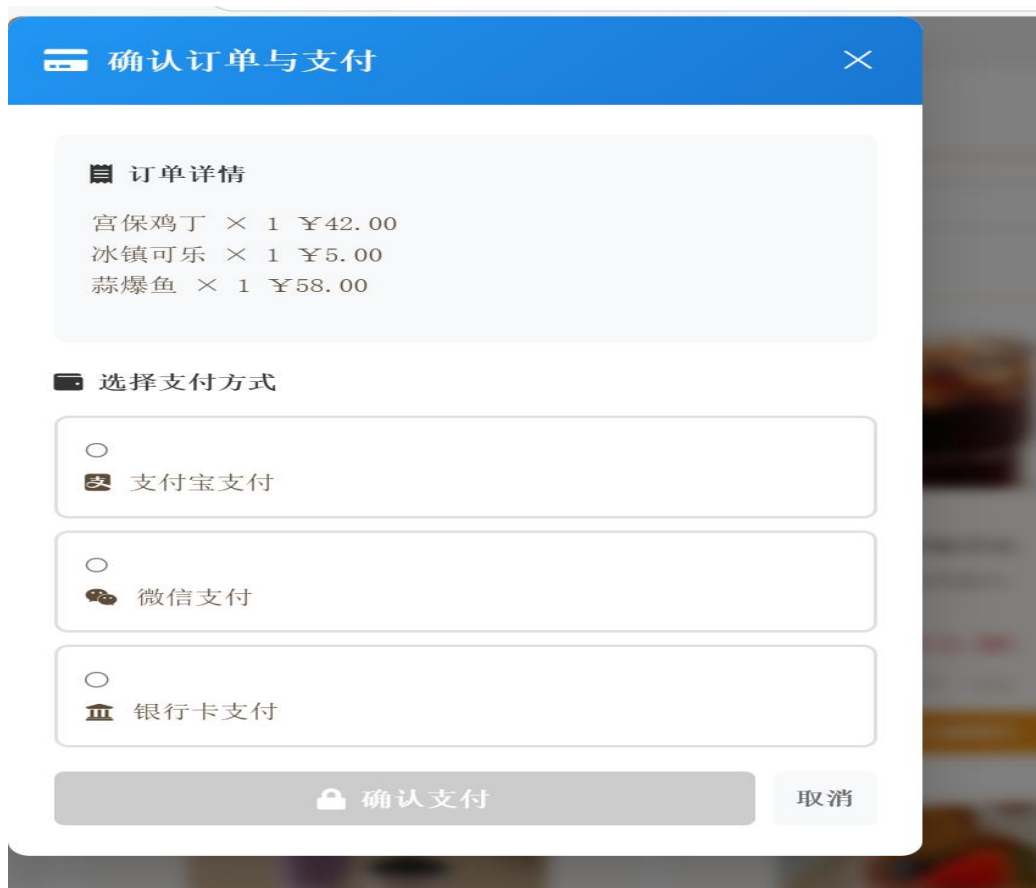


图 36 顾客支付页面展示图

3.1.4 数据库连接部分

1. 数据库连接测试:

```
<h2>📁 数据库连接测试</h2>
<%
// 修复后的数据库连接参数
String url = "jdbc:sqlserver://localhost:1433;databaseName=RestaurantDB;encrypt=false;trustServerCertificate=true";
String username = "sa";
String password = "20051029";

Connection connection = null;
Statement statement = null;
ResultSet resultSet = null;

try {
// 1. 加载JDBC驱动
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
%>
<div class="success">☑ JDBC驱动加载成功</div>
<%

// 2. 建立数据库连接
connection = DriverManager.getConnection(url, username, password);
%>
<div class="success">☑ 数据库连接成功! </div>
<%
}
```

图 1 数据库连接测试 jsp 代码

这是我的第一次JSP编程 - 数据库连接测试

hello 这是第0次输出
这是第1次输出
这是第2次输出
这是第3次输出
这是第4次输出
这是第5次输出
这是第6次输出
这是第7次输出
这是第8次输出
这是第9次输出
这是第10次输出

📁 数据库连接测试

- ☑ JDBC驱动加载成功
- ☑ 数据库连接成功!
- ☑ 数据表检查/创建完成
- ☑ 插入 3 条示例数据

📄 用户数据列表

ID	姓名	邮箱	年龄	创建时间
1	张三	zhangsan@email.com	25	2025-12-15 21:29:18.147
2	李四	lisi@email.com	30	2025-12-15 21:29:18.16
3	王五	wangwu@email.com	28	2025-12-15 21:29:18.163
总计: 3 条记录				

图 2 数据库连接测试结果

2. 登录页与数据库连接:

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="jspsqlserverdemo.DBConnection" %>
<%@ page import="java.sql.*, java.util.*, javax.servlet.http.*, javax.naming.*, javax.sql.DataSource" %>
<%@ page import="javax.xml.parsers.*, org.w3c.dom.*, java.io.*" %>
<%@ page import="jspsqlserverdemo.Appconfig" %>
<%
// 设置字符编码
request.setCharacterEncoding("UTF-8");
response.setCharacterEncoding("UTF-8");
response.setContentType("text/html;charset=UTF-8");

// 数据库连接测试结果变量
boolean dbConnected = false;
String dbMessage = "";
int dishCount = 0;
String driverInfo = "";
String urlInfo = "";

Connection conn = null;
Statement stmt = null;
ResultSet rs = null;
PreparedStatement pstmt = null;

try {
    // 读取配置文件
    String configPath = application.getRealPath("/WEB-INF/config.xml");
    AppConfig xmlconfig = new AppConfig(configPath);

    driverInfo = xmlconfig.getDriver();
    urlInfo = xmlconfig.getUrl();

    // 测试实际连接
    conn = DBConnection.getConnection();
    dbConnected = true;

    // 测试查询
    stmt = conn.createStatement();
    rs = stmt.executeQuery("SELECT COUNT(*) as count FROM Dishes");
    if (rs.next()) {
        dishCount = rs.getInt("count");
    }

    dbMessage = "数据库连接成功! <br>";
    dbMessage += "☑ 驱动: " + driverInfo + "<br>";
    dbMessage += "☑ URL: " + urlInfo + "<br>";
}

```

图 3 登录页连接数据库 jsp 代码

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>御膳房 - 登录</title>
  <link rel="stylesheet" href="css/登录页.css">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome/6.4.0/css/all.min.css">
  <link href="https://fonts.googleapis.com/css2?family=Noto+Sans+SC:wght@300;400;500;700&display=swap" rel="stylesheet">
</head>
<body>
<div id="dbModalOverlay" class="db-modal-overlay" style="display: none;"></div>
<div id="dbModal" class="db-modal" style="display: none;">
  <div class="db-modal-content">
    <div class="db-icon">
      <!-- 这里根据实际状态显示 <input checked="" type="checkbox"> 或 <input type="checkbox"> -->
      <%= dbConnected ? "<input checked="" type="checkbox">" : "<input type="checkbox">" %>
    </div>
    <div class="db-title">
      <!-- 这里根据实际状态显示 -->
      <%= dbConnected ? "数据库连接成功" : "数据库连接失败" %>
    </div>
    <div class="db-message">
      <!-- 这里显示详细消息 -->
      <%= dbMessage %>
    </div>
    <button id="dbCloseBtn" class="db-close-btn">
      <!-- 按钮文字根据状态变化 -->
      <%= dbConnected ? "开始使用系统" : "检查配置后重试" %>
    </button>
  </div>
</div>
<!-- 导航栏 -->
<nav class="navbar">
  <div class="navbar-container">
    <div class="navbar-logo">
      <i class="fas fa-utensils"></i>
      <span>御膳房</span>
    </div>
  </div>
</nav>
```

图 4 登录页连接数据库页面 jsp 代码

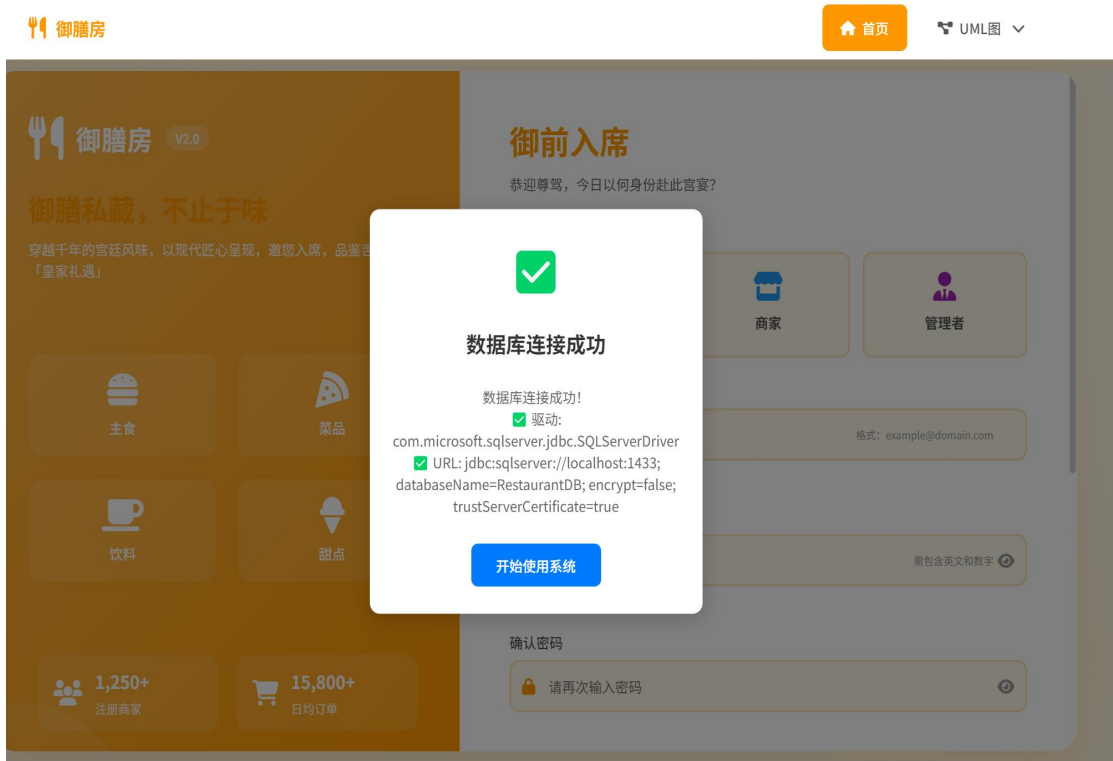


图 5 登录页连接数据库结果

3. 售卖页与数据库连接:

```
// 数据库连接测试结果变量
boolean dbConnected = false;
String dbMessage = "";
int dishCount = 0;
String driverInfo = "";
String urlInfo = "";

Connection conn = null;
Statement stmt = null;
ResultSet rs = null;
PreparedStatement pstmt = null;

try {
    // 读取配置文件
    String configPath = application.getRealPath("/WEB-INF/config.xml");
    AppConfig xmlconfig = new AppConfig(configPath);

    driverInfo = xmlconfig.getDriver();
    urlInfo = xmlconfig.getUrl();

    // 测试实际连接
    conn = DBConnection.getConnection();
    dbConnected = true;

    // 测试查询
    stmt = conn.createStatement();
    rs = stmt.executeQuery("SELECT COUNT(*) as count FROM Dishes");
    if (rs.next()) {
        dishCount = rs.getInt("count");
    }

    dbMessage = "数据库连接成功! <br>";
    dbMessage += "☑ 驱动: " + driverInfo + "<br>";
    dbMessage += "☑ URL: " + urlInfo + "<br>";
    dbMessage += "☑ 菜品记录数: " + dishCount + "条<br>";
    dbMessage += "☑ 每页显示: " + xmlconfig.getPageSize() + "条记录";
} catch (Exception e) {
    dbMessage = "✗ 数据库连接失败<br>";
    dbMessage += "错误信息: " + e.getMessage() + "<br>";
    dbMessage += "请检查: <br>";
    dbMessage += "1. SQL Server服务是否启动<br>";
    dbMessage += "2. 防火墙1433端口是否开放<br>";
    dbMessage += "3. 数据库用户名密码是否正确<br>";
    dbMessage += "4. 数据库是否存在<br>";
    e.printStackTrace();
}
```

图 6 售卖页连接数据库 jsp 文件

```
<body>
<div id="dbModalOverlay" class="db-modal-overlay" style="display: none;"></div>
<div id="dbModal" class="db-modal" style="display: none;">
    <div class="db-modal-content">
        <div class="db-icon">
            <!-- 这里根据实际状态显示 ☑ 或 ✗ -->
            <%= dbConnected ? "☑" : "✗" %>
        </div>
        <div class="db-title">
            <!-- 这里根据实际状态显示 -->
            <%= dbConnected ? "数据库连接成功" : "数据库连接失败" %>
        </div>
        <div class="db-message">
            <!-- 这里显示详细信息 -->
            <%= dbMessage %>
        </div>
        <button id="dbCloseBtn" class="db-close-btn">
            <!-- 按钮文字根据状态变化 -->
            <%= dbConnected ? "开始使用系统" : "检查配置后重试" %>
        </button>
    </div>
</div>
<!-- 页头: 用户信息和系统导航 -->
<header>
    <h1>欢迎使用餐饮管理系统</h1>
    <div class="user-info">
        <span id="currentUser">欢迎, 顾客</span> |
        <a href="admin.html" id="switchToAdmin">切换到管理页</a> |
        <a href="javascript:void(0);" onclick="logout()">退出登录</a>
    </div>
</header>
<!-- 点餐流程步骤指示器 -->
```

图 7 售卖页连接数据库页面 jsp 文件

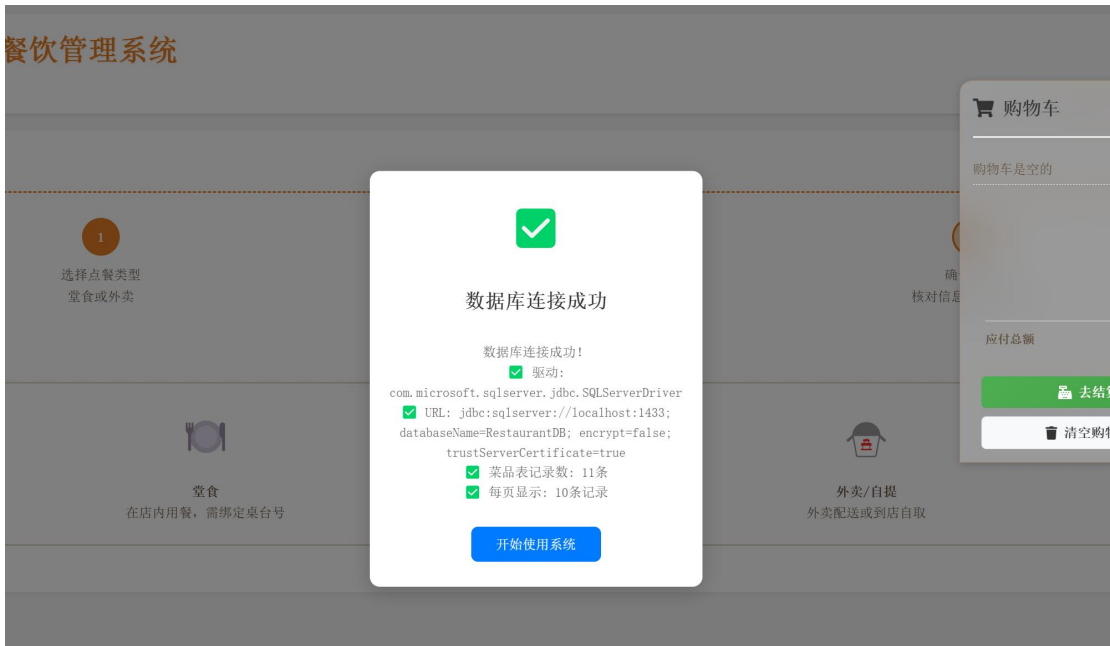


图 8 售卖页连接数据库结果

4. 通过 html 对数据库菜品增删改查：

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2 <%@ page import="java.sql.*" %>
3 <%@ page import="jspsqlserverdemo.DBConnection" %>
4 <%
5 // 测试数据库连接
6 boolean dbConnected = false;
7 try {
8     Connection testConn = DBConnection.getConnection();
9     dbConnected = testConn != null && !testConn.isClosed();
10    testConn.close();
11 } catch (Exception e) {
12    dbConnected = false;
13 }
14 %>
```

图 9 增删改查 jsp 文件

```

<body>
<h1>☐ 餐饮管理系统 - SQL Server 2008</h1>

<!-- 数据库连接状态显示 -->
<div class="db-status" <%= dbConnected ? "success" : "error" %>>
<%= dbConnected ? "☑ 成功连接到SQL Server 2008数据库" : "✗ 数据库连接失败" %>
</div>

<!-- 增删改查操作面板 -->
<div class="operation-panel">
<!-- 1. 增加数据 -->
<div class="panel-box">
<h3>➕ 新增菜品</h3>
<input type="text" id="addDishName" placeholder="菜品名称">
<input type="number" id="addDishPrice" placeholder="价格" step="0.01">
<input type="number" id="addDishStock" placeholder="库存数量">
<button onclick="addDish()">添加菜品</button>
</div>

<!-- 2. 查询数据 -->
<div class="panel-box">
<h3>🔍 查询菜品</h3>
<input type="text" id="searchKeyword" placeholder="输入关键词">
<select id="searchCategory">
<option value="">所有类别</option>
<option value="主食">主食</option>
<option value="饮料">饮料</option>
<option value="甜点">甜点</option>
</select>
<button onclick="searchDishes()">查询</button>
</div>

<!-- 3. 更新数据 -->
<div class="panel-box">
<h3>✎ 更新菜品</h3>
<input type="number" id="updateDishId" placeholder="菜品ID">
<input type="number" id="updatePrice" placeholder="新价格">
<input type="number" id="updateStock" placeholder="新库存">
<button onclick="updateDish()">更新菜品</button>
</div>

<!-- 4. 删除数据 -->
<div class="panel-box">

```

图 10 增删改查页面 jsp 文件

```

菜品处理.sql - 未连接  SQLQuery3.sql - LE...aurantDB (sa (58))  SQLQuery2.sql - 未连接  菜品.sql - LENOVO-M...rantDB (sa (5
CREATE DATABASE RestaurantDB
USE RestaurantDB

-- 创建Dishes表
CREATE TABLE Dishes (
    DishID INT IDENTITY(1,1) PRIMARY KEY,      -- 菜品ID, 主键, 自增长
    DishName NVARCHAR(100) NOT NULL,           -- 菜品名称
    Category NVARCHAR(50) NOT NULL,             -- 菜品分类
    Price DECIMAL(10,2) NOT NULL,              -- 菜品价格
    Cost DECIMAL(10,2) NOT NULL,               -- 菜品成本
    Stock INT NOT NULL DEFAULT 0,              -- 库存数量
    Description NVARCHAR(500) NULL             -- 菜品描述
);

-- 插入第一张图片中的菜品 (DishID 自动生成)
INSERT INTO Dishes (DishName, Category, Price, Cost, Stock, Description)
VALUES
('宫保鸡丁', '热菜', 42.00, 18.00, 50, '经典川菜, 香辣可口'),
('冰镇可乐', '饮品', 5.00, 1.50, 100, '畅爽酷饮'),
('蒜爆鱼', '热菜', 58.00, 25.00, 30, '蒜香浓郁, 鱼肉鲜嫩微辣'),
('米饭', '主食', 2.00, 0.50, 200, '洁白软糯, 谷物清甜'),
('羽衣甘蓝爆柠茶', '饮品', 15.00, 5.00, 40, '清爽微酸, 带有独特草本风味');

-- 插入第二张图片中的菜品 (DishID 自动生成)
INSERT INTO Dishes (DishName, Category, Price, Cost, Stock, Description)
VALUES
('芋泥奶茶', '饮品', 12.00, 4.00, 60, '绵密香芋与醇厚奶茶融合'),
('可可草莓松饼', '甜点', 18.00, 7.00, 35, '焦糖饼干点缀秋季甜蜜蛋糕塔'),
('蒜蓉虾', '热菜', 48.00, 20.00, 25, '蒜香浓郁, 虾肉弹嫩'),
('清蒸大闸蟹', '海鲜', 90.00, 40.00, 20, '鲜美清甜, 原汁原味'),
('双椒牛肉', '热菜', 52.00, 22.00, 30, '鲜辣过瘾, 牛肉滑嫩');

-- 查看 Dishes 表的所有数据
SELECT * FROM Dishes;

```

图 11 增删改查页面 sql 文件

餐饮管理系统 - SQL Server 2008

成功连接到SQL Server 2008数据库

新增菜品

菜品名称

价格

库存数量

添加菜品

查询菜品

芋泥

所有类别

查询

更新菜品

菜品ID

新价格

新库存

更新菜品

删除菜品

菜品ID

删除菜品

ID	菜品名称	分类	价格	成本	库存	描述
23	芋泥奶茶	饮品	¥12.0	¥4.0	60	绵密香芋与醇厚奶茶融合

未找到符合条件的菜品

图 12 增删改查网页结果

	DishID	DishName	Category	Price	Cost	Stock	Description	ImagePath	IsRecommended	Status	Created Time	Modified Time
1	18	宫保鸡丁	热菜	42.00	18.00	50	经典川菜，香辣可口	NULL	0	active	2025-12-17 01:36:52.573	2025-12-17 01:36:52.573
2	19	冰镇可乐	饮品	5.00	1.50	100	畅爽酷饮	NULL	0	active	2025-12-17 01:36:52.573	2025-12-17 01:36:52.573
3	20	蒜爆鱼	热菜	58.00	25.00	30	蒜香浓郁，鱼肉鲜嫩微辣	NULL	0	active	2025-12-17 01:36:52.573	2025-12-17 01:36:52.573
4	21	米饭	主食	2.00	0.50	200	洁白软糯，谷物清甜	NULL	0	active	2025-12-17 01:36:52.573	2025-12-17 01:36:52.573
5	22	羽衣甘蓝爆柠茶	饮品	15.00	5.00	40	清爽微酸，带有独特草本风味	NULL	0	active	2025-12-17 01:36:52.573	2025-12-17 01:36:52.573
6	23	芋泥奶茶	饮品	12.00	4.00	60	绵密香芋与醇厚奶茶融合	NULL	0	active	2025-12-17 01:36:52.577	2025-12-17 01:36:52.577
7	24	可可草莓松饼	甜点	18.00	7.00	35	焦糖饼干点缀秋季甜蜜蛋糕塔	NULL	0	active	2025-12-17 01:36:52.577	2025-12-17 01:36:52.577
8	25	蒜香虾	热菜	48.00	20.00	25	蒜香浓郁，虾肉弹嫩	NULL	0	active	2025-12-17 01:36:52.577	2025-12-17 01:36:52.577
9	26	清蒸大闸蟹	海鲜	90.00	40.00	20	鲜美清甜，原汁原味	NULL	0	active	2025-12-17 01:36:52.577	2025-12-17 01:36:52.577
10	27	双椒牛肉	热菜	52.00	22.00	30	鲜辣过瘾，牛肉滑嫩	NULL	0	active	2025-12-17 01:36:52.577	2025-12-17 01:36:52.577
11	28	可乐	默认分类	3.00	1.20	22	新添加的菜品	NULL	0	active	2025-12-17 02:21:15.723	2025-12-17 02:21:15.723

图 13 增删改查页面 sql 结果

3.2 结果展示

3.2.1 UML 用例图展示

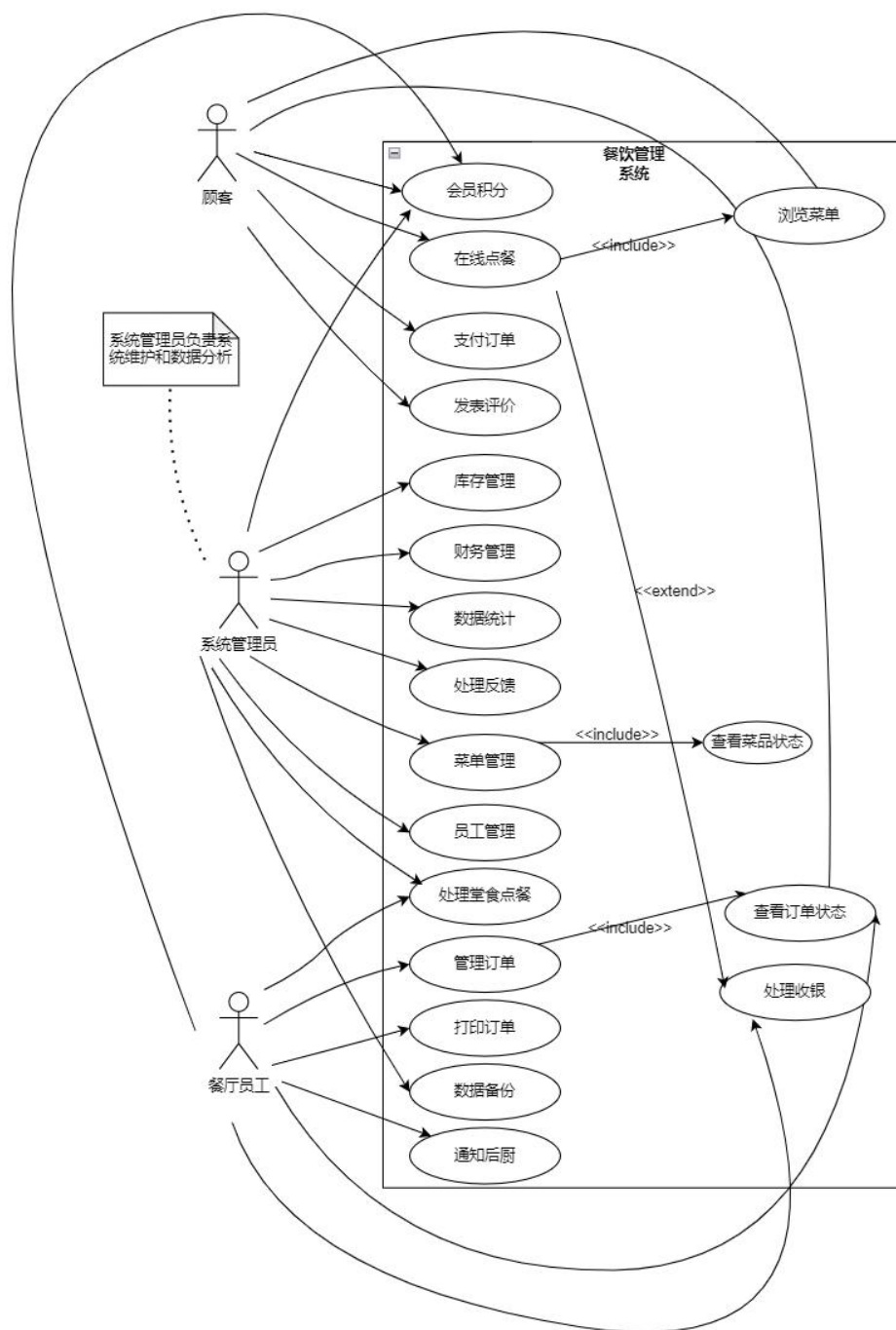


图 1 UML 的用例图展示

3.2.2 UML 的类图展示

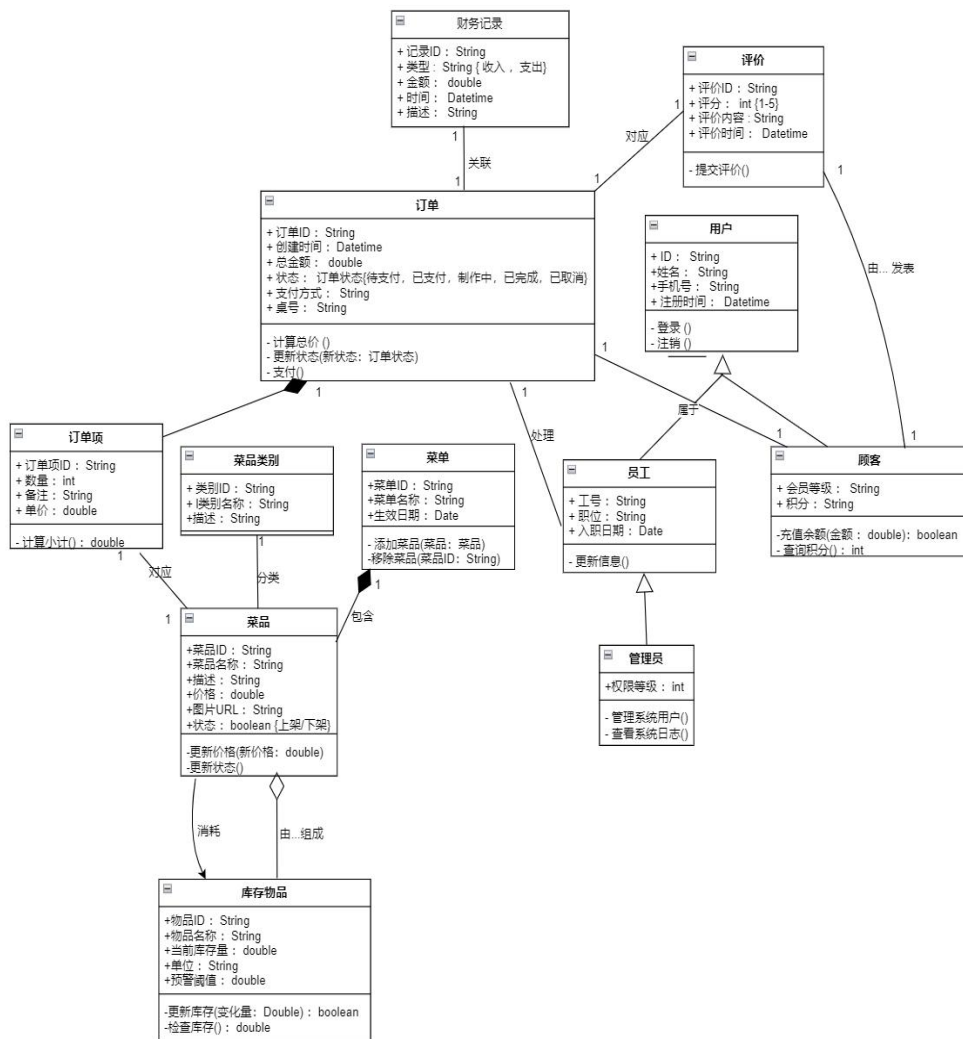


图 2 UML 的类图展示

3.2.3 UML 的活动图展示

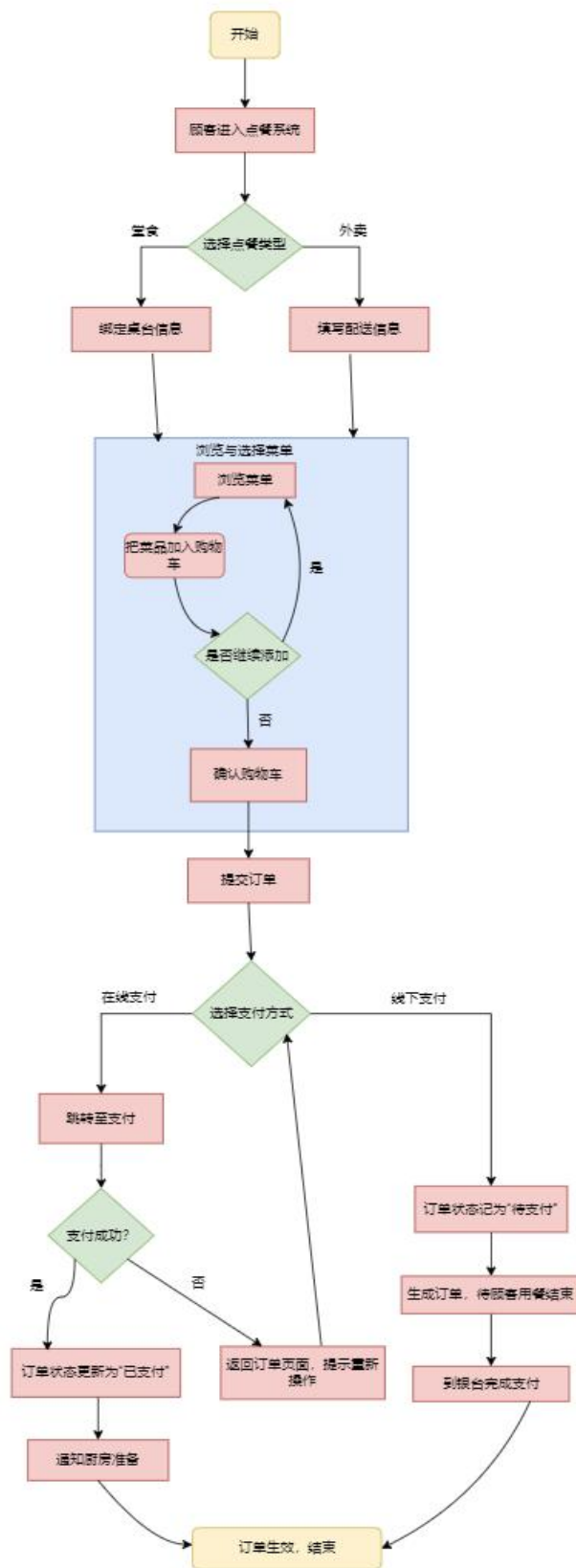


图 3 UML 的活动图 1 展示

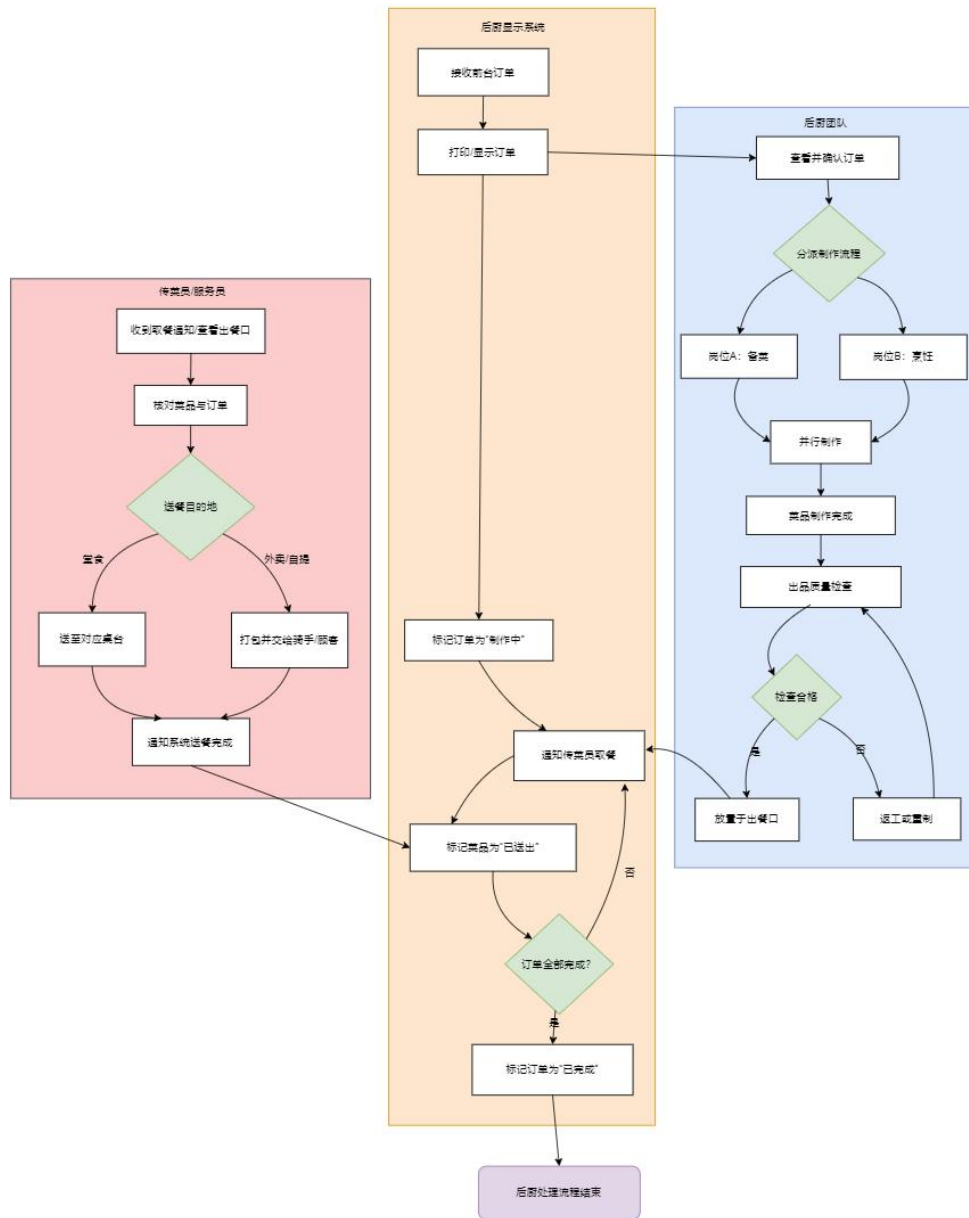


图 4 UML 的活动图 2 展示

四、餐饮管理系统的使用说明

该管理系统包含：顾客服务层：在线点餐、支付结算、评价互动；业务运营层：订单管理、桌台管理、厨房分单、收银处理；管理控制层：菜单管理、库存控制、财务管理、数据分析；系统支持层：用户权限管理、数据备份、系统维护。

前台点餐流程：

开始 → 选择点餐类型 → 填写信息（桌台/配送） → 浏览菜单 → 加入购物车 → 确认订单 → 选择支付方式 → （在线支付→跳转支付→支付成功→通知厨房） / （线下支付→用餐结束支付） → 订单生效

后厨处理流程：

前台订单 → 后厨显示系统接收 → 打印/显示订单 → 后厨团队分派制作 → 菜品制作与质检 → 合格菜品放置出餐口 → 传菜员核对取餐 → 按类型配送（堂食/外卖） → 系统标记完成

餐饮管理系统使用说明书

本说明书旨在指导用户理解并有效使用基于 **WEB** 架构的餐饮管理系统。系统通过用例图、类图及活动流程图的科学设计，实现了顾客、餐厅员工与系统管理员三大角色的协同作业，全面覆盖了点餐、备餐、结账与管理的完整业务流程。本系统通过数字化与自动化手段，旨在提升餐厅运营效率、优化顾客体验并实现精细化管理。

系统采用实时表单验证技术，邮箱输入框自动校验格式有效性。当用户输入不符合标准邮箱格式时，系统即时显示红色错误提示，并在输入框旁提供格式参考。

密码验证机制创新性地结合了前端 **JavaScript** 正则表达式验证与视觉化强度提示。系统不仅要求密码同时包含英文和数字，还通过动态强度条实时显示密码安全性等级（弱/中/强），并用颜色编码（红/黄/绿）直观反馈。

登录验证成功后，系统根据用户角色智能跳转到不同界面：顾客进入点餐页面，商家和管理员进入管理后台。"游客模式"设计，允许未注册用户浏览菜单但限制下单。当用户不存在时，系统不仅提示注册，还会自动切换到注册页面并预填已输入信息。除此之外，系统支持微信、支付宝、电话登录。

点击登录之后，自动连接餐饮数据库信息，若无该用户信息，则弹窗提醒用

户注册，并清除所有已填写内容。若能查到该用户信息，则直接跳转到餐饮售卖界面，并在跳转的同时通过 JDBC 完成与数据库的连接，若链接成功则弹窗“数据库连接成功”字样，否者弹窗“数据库连接失败”字样。

一、 主要角色与核心功能概述

系统依据用例图明确了三类核心用户及其权限：

1. 顾客：可通过系统在线点餐、支付订单、浏览菜单、发表评价及享受会员积分服务。顾客的在线操作流程清晰直观，如流程图所示，从选择点餐类型（堂食/外卖）到最终完成支付，系统均提供明确引导。

2. 餐厅员工：主要处理线下服务环节，包括处理堂食点餐、管理订单、打印订单、通知后厨及处理收银。员工是连接前台顾客与后厨生产的枢纽。

3. 系统管理员：拥有最高权限，负责后台的综合管理，包括菜单管理、员工管理、库存管理、财务管理、数据统计及系统维护，确保系统稳定运行并驱动业务决策。

二、 核心业务流程说明

1. 顾客在线点餐流程：

① 顾客进入系统后，首先选择“堂食”或“外卖”。如流程图所示，选择“堂食”需绑定桌台号，选择“外卖”则需填写配送信息。

② 随后，顾客可浏览菜单并将心仪菜品加入购物车，确认后提交订单。

③ 支付环节支持“在线支付”与“线下支付”。选择在线支付并成功后，订单状态自动更新为“已支付”，并通知后厨开始准备；选择线下支付，则订单状态为“待支付”，待用餐结束后前台结账。

2. 后厨订单处理与出餐流程：

① 前台订单会实时显示在后厨显示系统上。后厨团队确认订单后，按流程分派至各岗位（如备菜、烹饪）。

② 菜品制作完成后需经出品质量检查，合格则放置于出餐口，并通知系统标记该菜品“制作完成”。

③ 传菜员收到通知后，从出餐口取餐，核对订单与菜品。根据订单类型，将菜品送至对应桌台（堂食）或打包交给骑手/顾客（外卖）。

④ 送餐完成后，在系统中标记菜品“已送出”。当订单包含的所有菜

品均标记为“已送出”后，系统自动将整个订单状态更新为“已完成”。

3.系统管理员（最高权限）：

核心权限：系统后台综合管理

①用户账户管理：添加/编辑/删除管理员和商家账户

权限分配：设置用户类型（管理员/商家）

状态控制：启用/禁用用户账户

②菜单维护：添加、编辑、删除菜品信息

价格管理：设置和调整菜品价格

分类管理：菜品分类体系管理（推荐菜品、主食、饮料等）

库存控制：设置库存数量和可用状态

菜品描述：完善菜品详细介绍

③经营分析：用户数、菜品数、订单数、营收统计

实时监控：今日订单、待处理订单、已完成订单

营收分析：今日营收和总营收统计

三、 系统数据结构与交互逻辑

系统的数据模型由类图精确定义，核心实体间关系清晰：

① 订单是核心业务对象，与顾客、订单项、财务记录直接关联。一个订单包含多个订单项，每个订单项对应一份菜品。

② 菜品与库存物品通过“消耗”关系关联，销售时自动扣减库存，实现进销存联动。

③ 用户（包括顾客、员工、管理员）可发表评价。管理员则管理菜单、员工等所有核心数据。

四、 关键特性与使用价值

① 流程自动化：从顾客下单、后厨分单、制作、出餐到结账，状态自动流转，减少人工沟通误差。

② 数据实时联动：前台销售、后厨制作与库存数据实时同步，为管理决策提供准确依据。

③ 权责清晰：通过严格的角色权限控制，确保各岗位人员各司其职，操作可追溯。

三、系统数据结构与交互逻辑

系统的数据模型由类图精确定义，核心实体间关系清晰：

① 订单是核心业务对象，与顾客、订单项、财务记录直接关联。一个订单包含多个订单项，每个订单项对应一份菜品。

② 菜品与库存物品通过“消耗”关系关联，销售时自动扣减库存，实现进销存联动。

③ 用户（包括顾客、员工、管理员）可发表评价。管理员则管理菜单、员工等所有核心数据。

四、关键特性与使用价值

① 流程自动化：从顾客下单、后厨分单、制作、出餐到结账，状态自动流转，减少人工沟通误差。

② 数据实时联动：前台销售、后厨制作与库存数据实时同步，为管理决策提供准确依据。

③ 权责清晰：通过严格的角色权限控制，确保各岗位人员各司其职，操作可追溯。

本系统将传统餐饮业务转化为高效、透明的数字化流程，建议用户根据岗位职责熟悉对应功能模块，以充分发挥系统价值，实现降本增效。

五、分数构成

本次作业打分由小组成员分和个人得分所决定，总分 100 分。

1. UML 图部分：

用例图（5 分）

类图（3 分）

活动图（5 分）

2. Power Designer 部分：

成功构建 CDM（5 分）

成功生成 sql 脚本等文件（5 分）

3. 网页制作：

Html（20 分）

JavaScript 完成度（27 分）

CSS（10 分）

4. JSP 连接前后端：

JDBC 实现前后端连接的各项操作（20 分），若无法完成各项操作或操作不完善，则一个操作-3 分。

六、致谢

在本次“基于 WEB 的餐饮管理系统”的设计与开发完成之际，我谨向所有给予我指导、支持与帮助的老师、同学及亲友表示最诚挚的感谢。首先，衷心感谢我的指导老师。从课题的选定、系统的需求分析、UML 建模，到后续的数据库设计、功能实现与测试，老师始终给予我耐心的指导和宝贵的建议。每当我在技术实现或逻辑设计中遇到瓶颈时，老师总能以渊博的知识和清晰的思路为我指点迷津，帮助我明确方向、理清架构。老师严谨的治学态度、精益求精的专业精神，不仅贯穿于本项目，更将使我受益终身。其次，感谢项目小组的各位同学。在系统分析与设计阶段，我们共同探讨业务流程、绘制图表；在编码实现与测试阶段，我们相互协助、解决技术难题。感谢大家营造了积极协作、互帮互助的团队氛围，正是集体的智慧与共同努力，才使得项目能够克服重重困难，顺利完成。同时，感谢我的家人和朋友。感谢他们在项目进行期间给予的理解、支持与鼓励，他们是我能够安心投入设计与开发工作的坚实后盾。最后，感谢所有为本项目提供参考与启发的文献作者、开源项目贡献者以及网络技术社区的分享者们。站在前人的肩膀上，我才得以更快地学习和应用新技术，完成系统构建。由于学识与经验有限，系统中难免存在疏漏与不足之处，恳请各位老师、同学批评指正。再次向所有关心、帮助我的人们致以最真诚的谢意！